

Laboratorio 10 - Aprendizaje Semi-Supervisado

Jose Sanchez, Roberto Najera

2026-05-15

Contents

Introducción	3
Actividad 1 — Selección y Análisis del Dataset	3
1.1 Selección y Descripción del Dataset	3
1.2 Análisis Exploratorio de Datos	3
1.2.1 Carga del Dataset	3
1.2.2 Tipos de Variables y Estructura	3
1.2.3 Resumen Estadístico	4
1.2.4 Valores Faltantes	5
1.2.5 Balance de Clases	6
1.2.6 Distribución de Variables Numéricas	7
1.2.7 Distribución de Variables Categóricas Clave	8
1.2.8 Variables Numéricas vs. Clase Objetivo	11
1.2.9 Correlación entre Variables Numéricas	12
1.3 Preprocesamiento de Datos	13
1.3.1 Limpieza: Eliminación de Valores Faltantes	13
1.3.2 Eliminación de <code>fnlwgt</code>	14
1.3.3 Codificación de la Variable Objetivo	14
1.3.4 Normalización de Variables Numéricas	14
1.3.5 Variables Categóricas: Verificación de Factores	15
1.3.6 Resumen del Dataset Preprocesado	15
Actividad 2 — Diseño Experimental Semi-Supervisado	16
2.1 Diseño del Escenario Semi-Supervisado	16
2.2 Función Auxiliar de Evaluación	18
2.3 Modelo Supervisado Baseline: Random Forest	19
2.3.1 Fundamento	19

2.3.2 Entrenamiento y Evaluación	19
2.3.3 Importancia de Variables	20
2.3.4 Sensibilidad a Hiperparámetros (ntree y mtry)	21
2.4 Algoritmo Semi-Supervisado 1: Self-Training	22
2.4.1 Fundamento Conceptual y Matemático	22
2.4.2 Implementación Manual	23
2.4.3 Evolución del Entrenamiento	25
2.4.4 Sensibilidad al Threshold τ	26
2.5 Algoritmo Semi-Supervisado 2: Co-Training	27
2.5.1 Fundamento Conceptual y Matemático	27
2.5.2 Implementación Manual	28
2.5.3 Sensibilidad al Parámetro k_top	30
2.6 Comparación Global de Modelos	32
2.7 Análisis de Sensibilidad al Porcentaje de Etiquetas	33
2.8 Comparación Real vs. Predicción	36
2.9 Discusión	37
2.10 Conclusiones	38
Actividad 3 — Experimentación e Hiperparámetros	38
3.1 Curvas de Aprendizaje (Sobreajuste vs. Subajuste)	38
3.2 Estabilidad Estocástica con Múltiples Seeds	40
3.3 Grid Search Exhaustivo — Self-Training	42
3.4 Grid Search Exhaustivo — Co-Training	44
3.5 Análisis de Sensibilidad Granular al % de Etiquetas	46
3.6 Significancia Estadística — Prueba de Wilcoxon	48
3.7 Resumen Experimental	49
Actividad 4 — Visualización y Análisis Gráfico	50
4.1 Matrices de Confusión Comparativas	50
4.2 Curvas ROC y Área Bajo la Curva (AUC)	51
4.3 Curvas Precisión–Recall	53
4.4 Distribución de Probabilidades Predichas por Clase Real	55
4.5 Evolución del Error de Self-Training por Iteración	56
4.6 Curvas de Desempeño vs. % de Etiquetas — Cuatro Métricas	58
4.7 Heatmap Comparativo de Métricas $\times$ Modelos	59
4.8 Comparación Real vs. Predicción — Muestra Individual	61
4.9 Resumen Visual del Análisis Gráfico	62

Introducción

El presente laboratorio aplica técnicas de **aprendizaje semi-supervisado** sobre el dataset *Adult Income* (UCI Machine Learning Repository). El objetivo es comparar modelos supervisados puros contra algoritmos semi-supervisados bajo un escenario donde solo una fracción pequeña de los datos de entrenamiento posee etiqueta.

El dataset fue recolectado a partir del Censo de los Estados Unidos de 1994 y contiene información socioeconómica de individuos. La variable objetivo indica si el ingreso anual de una persona supera o no los USD 50,000.

Actividad 1 — Selección y Análisis del Dataset

Objetivo: identificar un dataset real y público, caracterizarlo a profundidad mediante un análisis exploratorio, y dejarlo limpio y transformado para los algoritmos semi-supervisados. Esta actividad cubre la **selección del dataset (1.1)**, el **análisis exploratorio (1.2)** y el **pre-procesamiento (1.3)**.

1.1 Selección y Descripción del Dataset

Fuente: UCI Machine Learning Repository — *Adult Income Dataset* **URL:** <https://archive.ics.uci.edu/dataset/2/adult> **Dimensiones:** 48,842 filas × 15 columnas **Variable objetivo:** income — binaria ($\leq 50K$ / $> 50K$)

El dataset cumple con los requisitos mínimos del laboratorio: más de 1,000 filas, más de 8 columnas, fuente pública y confiable, y tarea de clasificación binaria apta para el escenario semi-supervisado.

1.2 Análisis Exploratorio de Datos

1.2.1 Carga del Dataset

```
df <- read.csv("adult_incomeDATA.csv",
               stringsAsFactors = TRUE,
               na.strings       = c("NA", ""))

cat("Dimensiones:", nrow(df), "x", ncol(df), "\n")
```

```
## Dimensiones: 48842 x 15
```

1.2.2 Tipos de Variables y Estructura

```
str(df)
```

```
## 'data.frame':    48842 obs. of  15 variables:
## $ age           : int   39 50 38 53 28 37 49 52 31 42 ...
## $ workclass      : Factor w/ 8 levels "Federal-gov",...: 7 6 4 4 4 4 6 4 4 ...
## $ fnlwgt         : int  77516 83311 215646 234721 338409 284582 160187 209642 45781 159449 ...
## $ education      : Factor w/ 16 levels "10th","11th",...: 10 10 12 2 10 13 7 12 13 10 ...
## $ education_num  : int   13 13 9 7 13 14 5 9 14 13 ...
## $ marital_status : Factor w/ 7 levels "Divorced","Married-AF-spouse",...: 5 3 1 3 3 3 4 3 5 3 ...
## $ occupation     : Factor w/ 14 levels "Adm-clerical",...: 1 4 6 6 10 4 8 4 10 4 ...
## $ relationship   : Factor w/ 6 levels "Husband","Not-in-family",...: 2 1 2 1 6 6 2 1 2 1 ...
## $ race           : Factor w/ 5 levels "Amer-Indian-Eskimo",...: 5 5 5 3 3 5 3 5 5 5 ...
## $ sex            : Factor w/ 2 levels "Female","Male": 2 2 2 2 1 1 1 2 1 2 ...
## $ capital_gain   : int   2174 0 0 0 0 0 0 0 14084 5178 ...
## $ capital_loss   : int    0 0 0 0 0 0 0 0 0 0 ...
## $ hours_per_week : int   40 13 40 40 40 40 16 45 50 40 ...
## $ native_country : Factor w/ 41 levels "Cambodia","Canada",...: 39 39 39 39 5 39 23 39 39 39 ...
## $ income         : Factor w/ 2 levels "<=50K",">50K": 1 1 1 1 1 1 1 2 2 2 ...
```

El dataset contiene dos tipos principales de variables:

Tipo	Variables
Numéricas (6)	age, fnlwgt, education_num, capital_gain, capital_loss, hours_per_week
Catóricas (9)	workclass, education, marital_status, occupation, relationship, race, sex, native_country, income

1.2.3 Resumen Estadístico

```
summary(df)
```

```
##           age           workclass           fnlwgt
## Min.      :17.00   Private          :33906   Min.      : 12285
## 1st Qu.:28.00   Self-emp-not-inc: 3862   1st Qu.: 117551
## Median :37.00   Local-gov          : 3136   Median : 178145
## Mean     :38.64   State-gov          : 1981   Mean    : 189664
## 3rd Qu.:48.00   Self-emp-inc       : 1695   3rd Qu.: 237642
## Max.     :90.00   (Other)            : 1463   Max.    :1490400
##              NA's              : 2799
##           education   education_num           marital_status
## HS-grad      :15784   Min.      : 1.00   Divorced          : 6633
## Some-college:10878   1st Qu.: 9.00   Married-AF-spouse :   37
## Bachelors    : 8025   Median :10.00   Married-civ-spouse :22379
## Masters      : 2657   Mean     :10.08   Married-spouse-absent: 628
## Assoc-voc    : 2061   3rd Qu.:12.00   Never-married     :16117
## 11th         : 1812   Max.     :16.00   Separated         : 1530
## (Other)      : 7625           Widowed          : 1518
##           occupation           relationship           race
## Prof-specialty : 6172   Husband          :19716   Amer-Indian-Eskimo: 470
## Craft-repair   : 6112   Not-in-family    :12583   Asian-Pac-Islander: 1519
## Exec-managerial: 6086   Other-relative   : 1506   Black              : 4685
```

```
## Adm-clerical : 5611 Own-child : 7581 Other : 406
## Sales : 5504 Unmarried : 5125 White :41762
## (Other) :16548 Wife : 2331
## NA's : 2809
## sex capital_gain capital_loss hours_per_week
## Female:16192 Min. : 0 Min. : 0.0 Min. : 1.00
## Male :32650 1st Qu.: 0 1st Qu.: 0.0 1st Qu.:40.00
## Median : 0 Median : 0.0 Median :40.00
## Mean : 1079 Mean : 87.5 Mean :40.42
## 3rd Qu.: 0 3rd Qu.: 0.0 3rd Qu.:45.00
## Max. :99999 Max. :4356.0 Max. :99.00
##
## native_country income
## United-States:43832 <=50K:37155
## Mexico : 951 >50K :11687
## Philippines : 295
## Germany : 206
## Puerto-Rico : 184
## (Other) : 2517
## NA's : 857
```

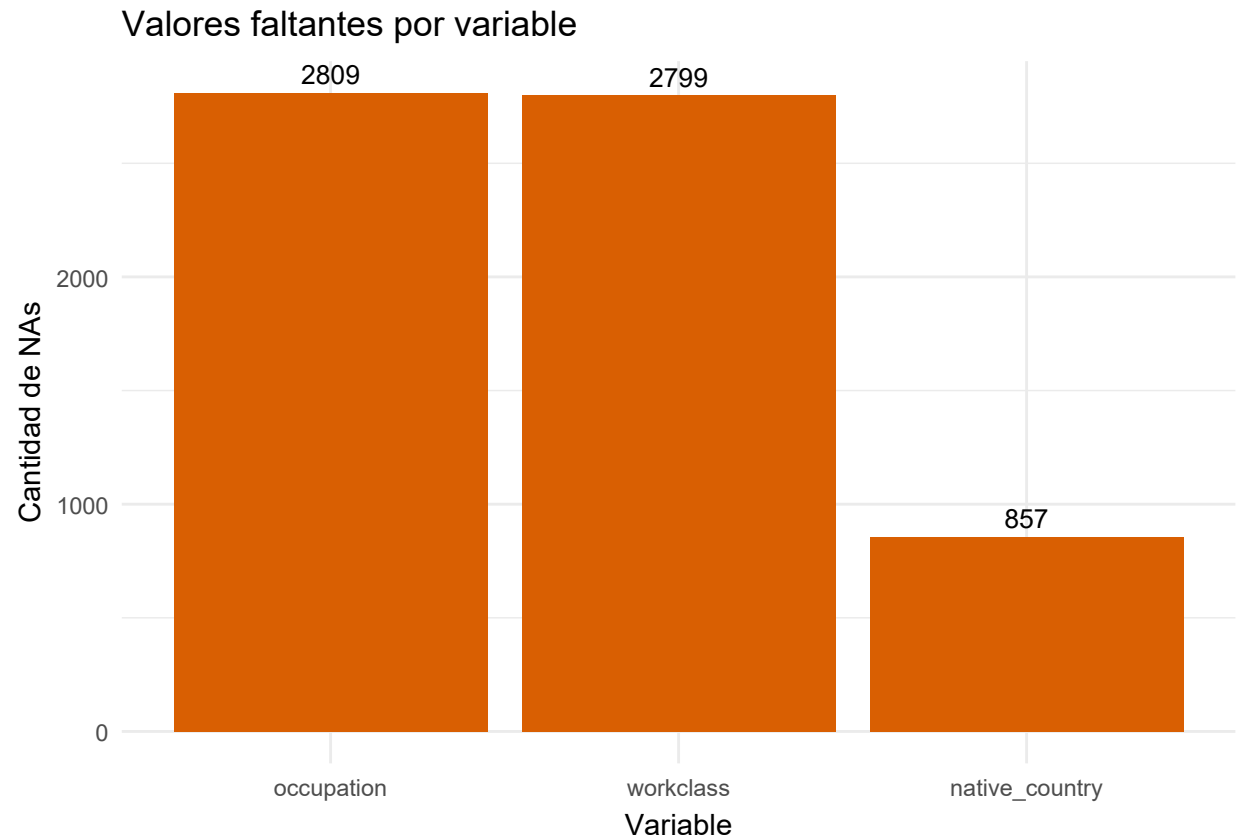
1.2.4 Valores Faltantes

```
na_conteo <- colSums(is.na(df))
na_df <- data.frame(variable = names(na_conteo),
                    faltantes = as.integer(na_conteo)) %>%
  filter(faltantes > 0) %>%
  arrange(desc(faltantes))

print(na_df)
```

```
## variable faltantes
## 1 occupation 2809
## 2 workclass 2799
## 3 native_country 857
```

```
ggplot(na_df, aes(x = reorder(variable, -faltantes), y = faltantes)) +
  geom_col(fill = "#d95f02") +
  geom_text(aes(label = faltantes), vjust = -0.4, size = 3.5) +
  labs(title = "Valores faltantes por variable",
       x = "Variable", y = "Cantidad de NAs") +
  theme_minimal()
```



Las variables con valores faltantes son `workclass` (2,799), `occupation` (2,809) y `native_country` (857), representando en conjunto un ~8% del total. Se optará por eliminación de filas con NA dado que el volumen de datos es suficientemente grande para absorber esta pérdida sin afectar la representatividad.

1.2.5 Balance de Clases

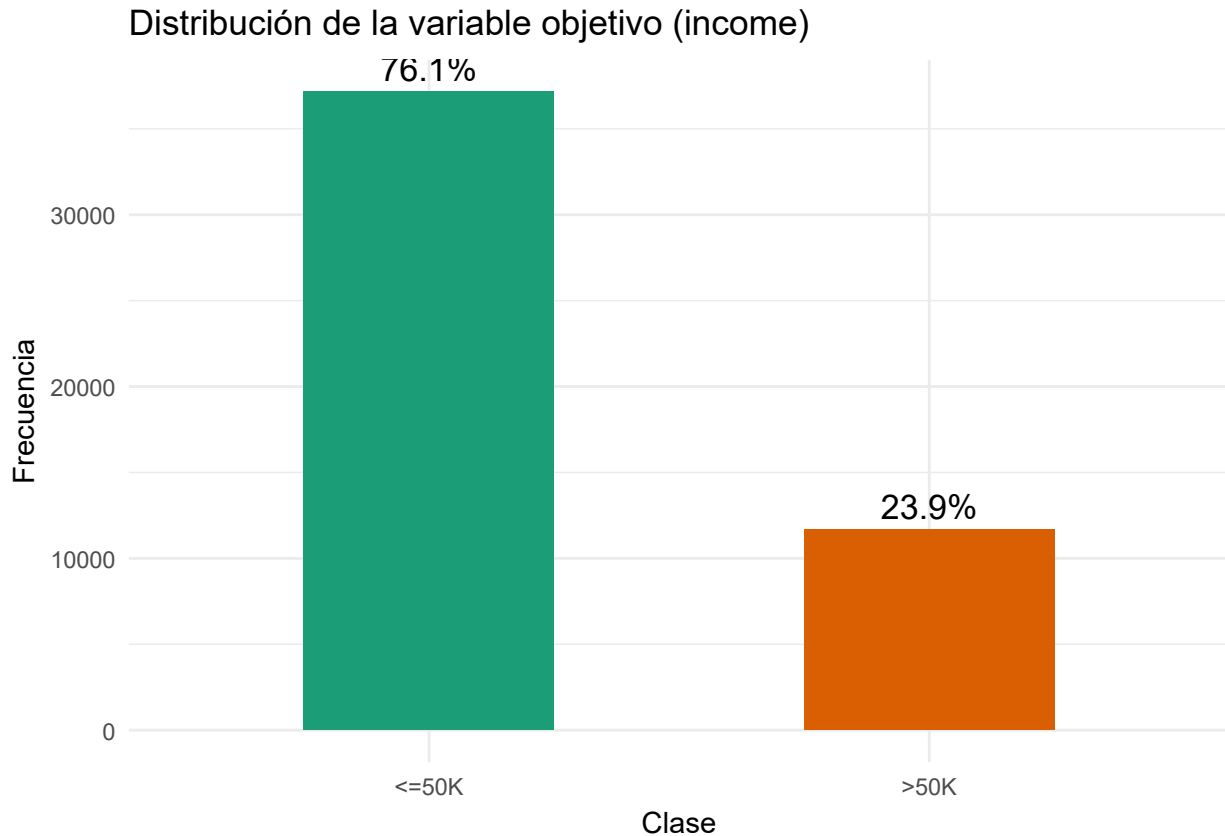
```
balance <- df %>%
  filter(!is.na(income)) %>%
  count(income) %>%
  mutate(porcentaje = n / sum(n) * 100)

print(balance)
```

```
##   income      n porcentaje
## 1  <=50K 37155   76.07182
## 2   >50K 11687   23.92818
```

```
ggplot(balance, aes(x = income, y = n, fill = income)) +
  geom_col(width = 0.5, show.legend = FALSE) +
  geom_text(aes(label = paste0(round(porcentaje, 1), "%")),
    vjust = -0.4, size = 4.5) +
```

```
scale_fill_manual(values = c("#1b9e77", "#d95f02")) +
labs(title = "Distribución de la variable objetivo (income)",
     x = "Clase", y = "Frecuencia") +
theme_minimal()
```



Existe **desbalance de clases**: el 75.9% de los individuos gana <=50K y el 24.1% gana >50K. Esto deberá considerarse al elegir métricas de evaluación (F1-score y recall son más informativas que accuracy en este caso) y al definir estrategias de muestreo en los modelos.

1.2.6 Distribución de Variables Numéricas

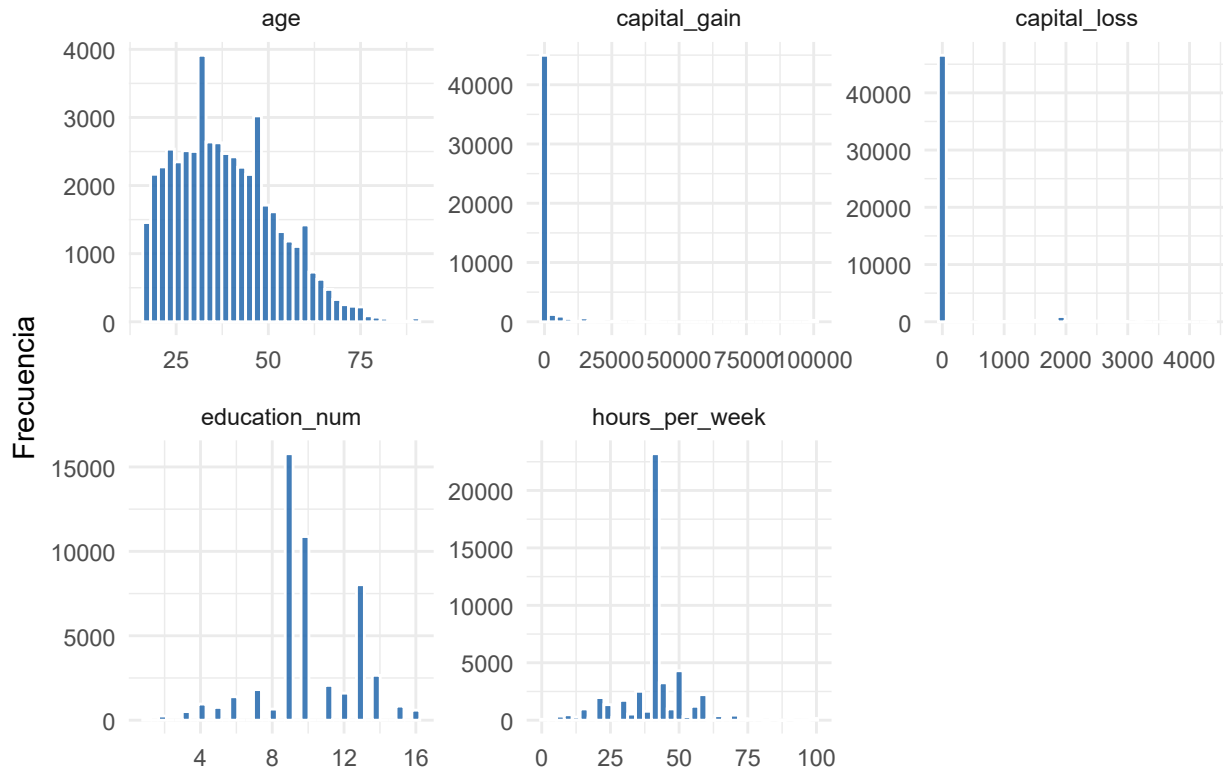
```
numericas <- df %>%
  select(age, education_num, capital_gain, capital_loss, hours_per_week)

numericas_long <- numericas %>%
  pivot_longer(cols = everything(), names_to = "variable", values_to = "valor")

ggplot(numericas_long, aes(x = valor)) +
  geom_histogram(bins = 35, fill = "#2166ac", color = "white", alpha = 0.85) +
  facet_wrap(~variable, scales = "free", ncol = 3) +
  labs(title = "Distribución de variables numéricas",
```

```
x = NULL, y = "Frecuencia") +  
theme_minimal()
```

Distribución de variables numéricas

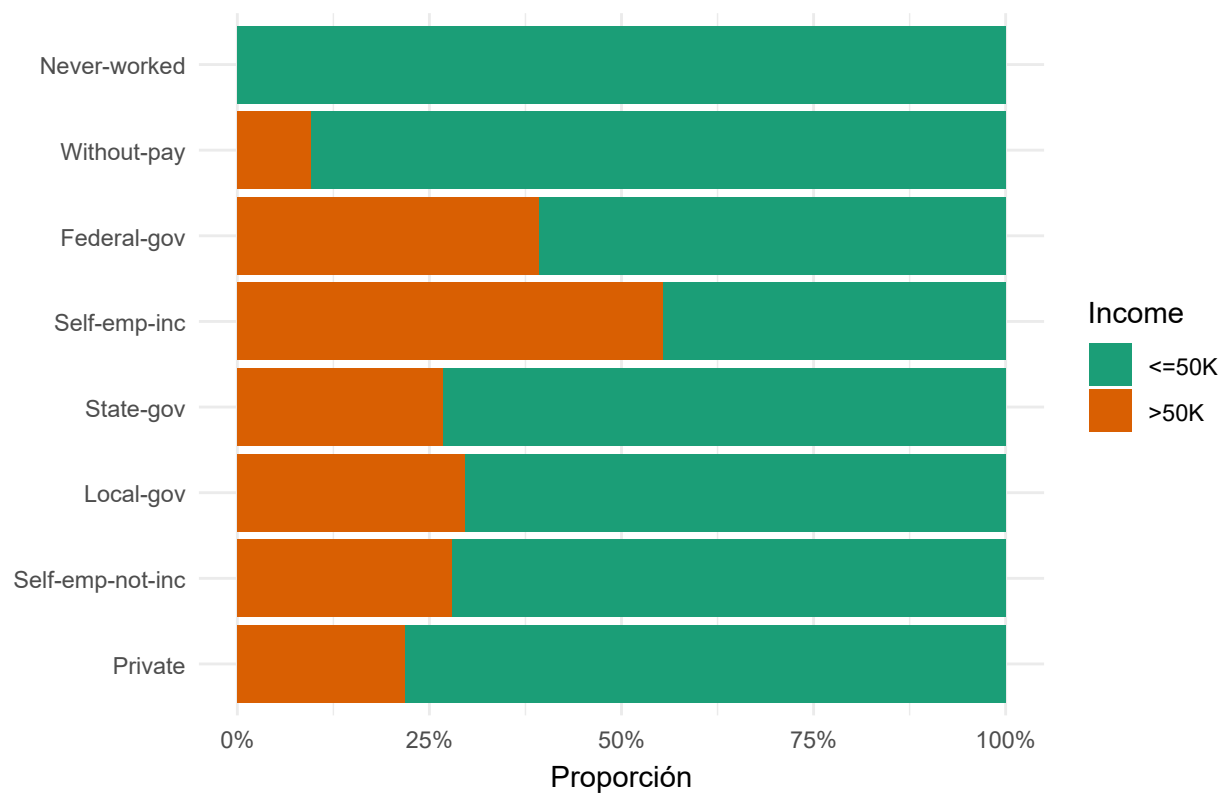


- **age:** distribución aproximadamente normal, concentrada entre 25–55 años.
- **education_num:** discreta; pico en niveles 9 (HS-grad) y 13 (Bachelors).
- **capital_gain / capital_loss:** fuertemente sesgadas a la derecha — la mayoría de individuos reporta cero ganancias/pérdidas de capital.
- **hours_per_week:** pico marcado en 40 horas, con cola derecha.

1.2.7 Distribución de Variables Categóricas Clave

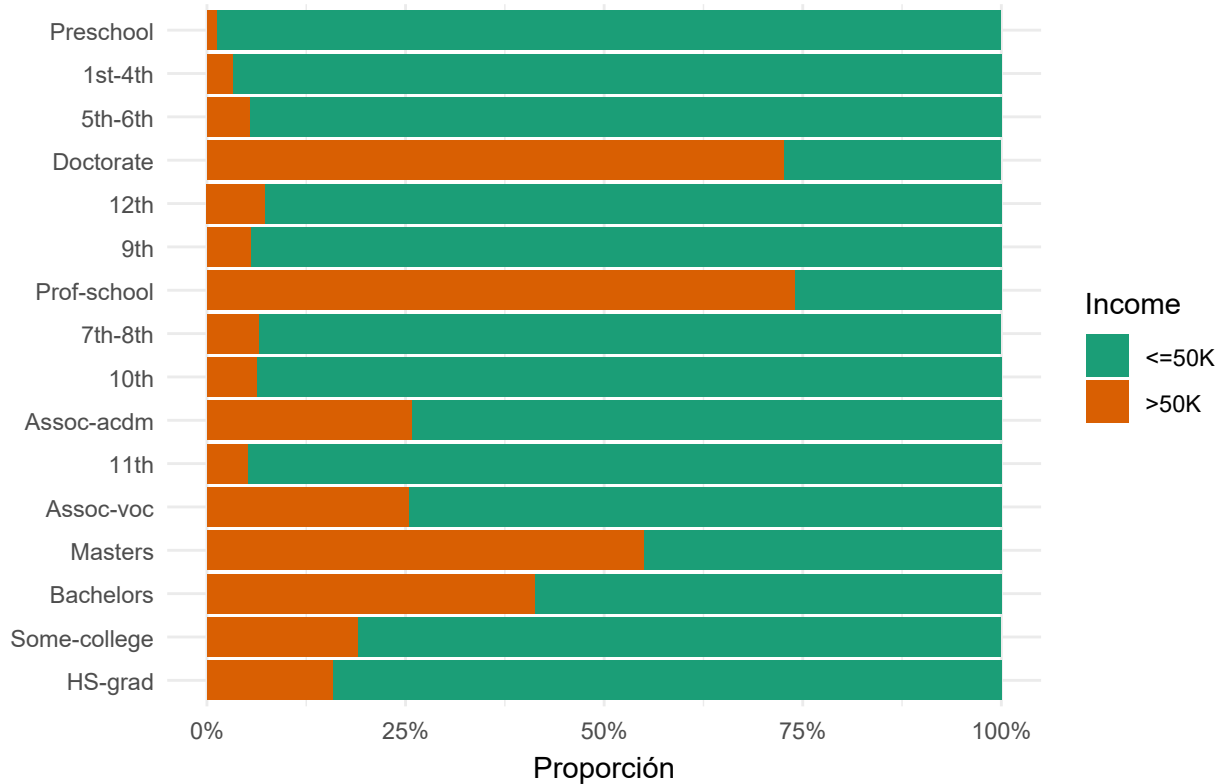
```
# Workclass  
ggplot(df %>% filter(!is.na(workclass)),  
  aes(x = fct_infreq(workclass), fill = income)) +  
  geom_bar(position = "fill") +  
  coord_flip() +  
  scale_y_continuous(labels = percent) +  
  scale_fill_manual(values = c("#1b9e77", "#d95f02")) +  
  labs(title = "Workclass vs. Income (proporción)",  
    x = NULL, y = "Proporción", fill = "Income") +  
  theme_minimal()
```


Workclass vs. Income (proporción)

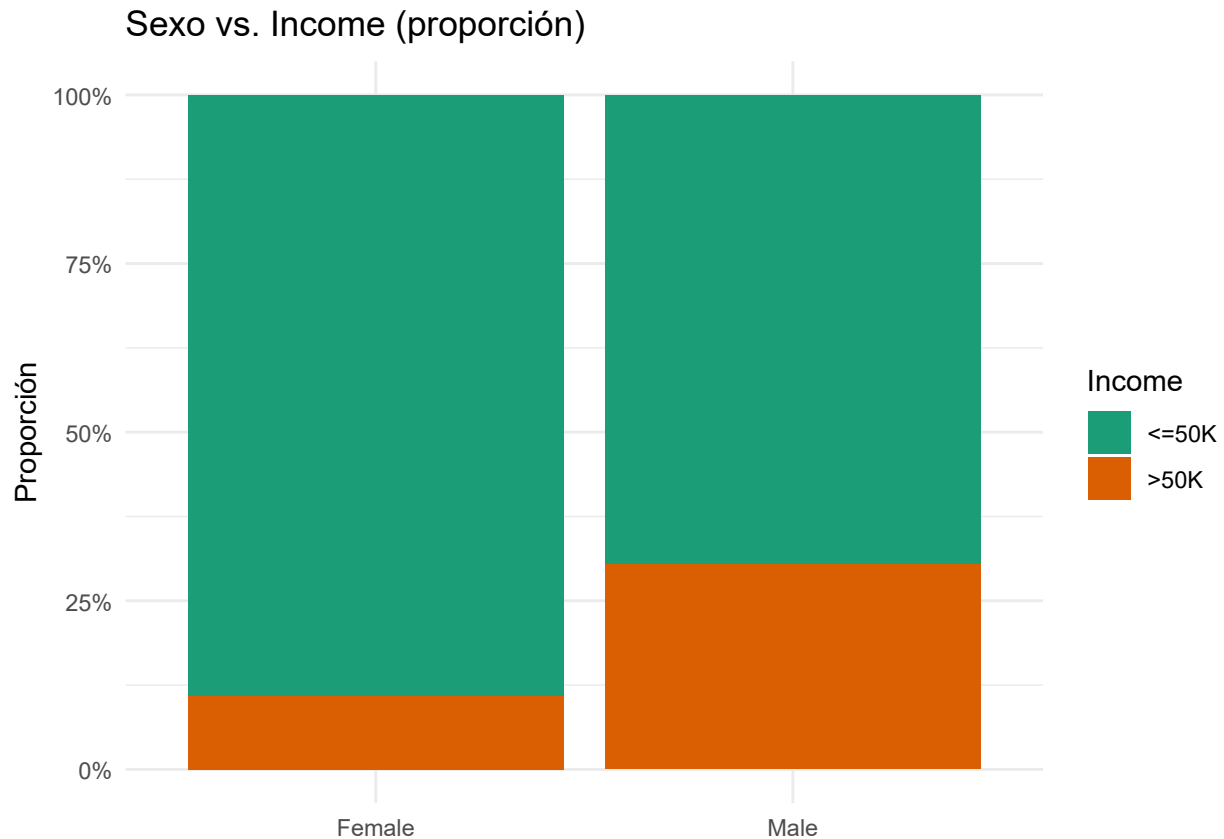


```
# Education
ggplot(df %>% filter(!is.na(education)),
  aes(x = fct_infreq(education), fill = income)) +
  geom_bar(position = "fill") +
  coord_flip() +
  scale_y_continuous(labels = percent) +
  scale_fill_manual(values = c("#1b9e77", "#d95f02")) +
  labs(title = "Nivel educativo vs. Income (proporción)",
    x = NULL, y = "Proporción", fill = "Income") +
  theme_minimal()
```

Nivel educativo vs. Income (proporción)



```
# Sex
ggplot(df, aes(x = sex, fill = income)) +
  geom_bar(position = "fill") +
  scale_y_continuous(labels = percent) +
  scale_fill_manual(values = c("#1b9e77", "#d95f02")) +
  labs(title = "Sexo vs. Income (proporción)",
       x = NULL, y = "Proporción", fill = "Income") +
  theme_minimal()
```



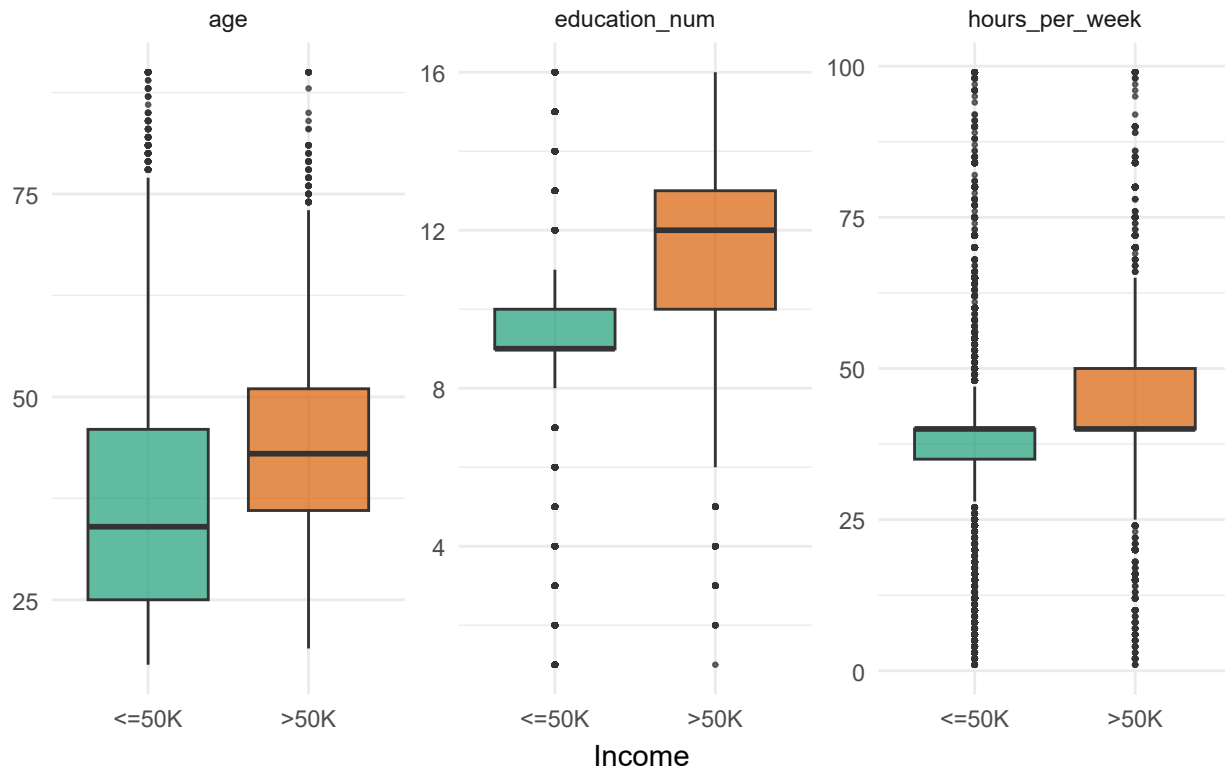
- El sector **Self-emp-inc** y **Federal-gov** concentran mayor proporción de ingresos >50K.
- A mayor nivel educativo (Doctorate, Prof-school, Masters), mayor proporción de >50K.
- Existe una diferencia notable entre sexos: los hombres presentan mayor tasa de >50K.

1.2.8 Variables Numéricas vs. Clase Objetivo

```
df_box <- df %>%
  select(income, age, education_num, hours_per_week) %>%
  filter(!is.na(income)) %>%
  pivot_longer(-income, names_to = "variable", values_to = "valor")

ggplot(df_box, aes(x = income, y = valor, fill = income)) +
  geom_boxplot(alpha = 0.7, outlier.size = 0.5, show.legend = FALSE) +
  facet_wrap(~variable, scales = "free_y") +
  scale_fill_manual(values = c("#1b9e77", "#d95f02")) +
  labs(title = "Variables numéricas según clase de ingreso",
       x = "Income", y = NULL) +
  theme_minimal()
```

Variables numéricas según clase de ingreso



Las personas con ingresos >50K tienden a ser de mayor edad, con más años de educación formal y más horas de trabajo semanales — lo que sugiere que estas variables tienen poder discriminativo relevante.

1.2.9 Correlación entre Variables Numéricas

```
num_vars <- df %>%
  select(age, fnlwgt, education_num, capital_gain, capital_loss, hours_per_week) %>%
  cor(use = "complete.obs")

num_df <- as.data.frame(as.table(num_vars))
names(num_df) <- c("Var1", "Var2", "Correlacion")

ggplot(num_df, aes(x = Var1, y = Var2, fill = Correlacion)) +
  geom_tile(color = "white") +
  geom_text(aes(label = round(Correlacion, 2)), size = 3) +
  scale_fill_gradient2(low = "#d73027", mid = "white", high = "#1a9641",
    midpoint = 0, limits = c(-1, 1)) +
  labs(title = "Matriz de correlación - variables numéricas",
    x = NULL, y = NULL) +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```

Matriz de correlación — variables numéricas



Las correlaciones entre variables numéricas son en general bajas, lo que indica poca multicolinealidad. `education_num` y `age` tienen una correlación modesta positiva con los ingresos altos (como ya se vio en los boxplots). `fnlwgt` (peso censal) no muestra relación relevante con otras variables.

1.3 Preprocesamiento de Datos

1.3.1 Limpieza: Eliminación de Valores Faltantes

```
df_clean <- na.omit(df)
cat("Filas originales :", nrow(df), "\n")
```

```
## Filas originales : 48842
```

```
cat("Filas tras omitir NAs:", nrow(df_clean), "\n")
```

```
## Filas tras omitir NAs: 45222
```

```
cat("Filas eliminadas :", nrow(df) - nrow(df_clean), "(",
    round((nrow(df) - nrow(df_clean)) / nrow(df) * 100, 1), "% )\n")
```

```
## Filas eliminadas : 3620 ( 7.4 % )
```

Se eliminan las 3,620 filas con al menos un NA (~7.4%), estrategia válida dado el amplio volumen de datos restante (45,222 filas).

1.3.2 Eliminación de fnlwgt

```
df_clean$fnlwgt <- NULL
cat("Columnas finales:", ncol(df_clean), "\n")
```

```
## Columnas finales: 14
```

La variable `fnlwgt` es un peso de muestreo censal que no aporta información predictiva sobre el ingreso individual. Se elimina del análisis.

1.3.3 Codificación de la Variable Objetivo

```
df_clean$income <- ifelse(df_clean$income == ">50K", 1L, 0L)
df_clean$income <- as.factor(df_clean$income)

cat("Distribución del target tras recodificación:\n")
```

```
## Distribución del target tras recodificación:
```

```
print(table(df_clean$income))
```

```
##
##      0      1
## 34014 11208
```

```
print(prop.table(table(df_clean$income)))
```

```
##
##           0           1
## 0.752156 0.247844
```

La variable `income` se convierte a factor binario: 1 para >50K y 0 para <=50K, lo que es requerido por los clasificadores de R.

1.3.4 Normalización de Variables Numéricas

```
num_cols <- c("age", "education_num", "capital_gain",
              "capital_loss", "hours_per_week")

df_clean[num_cols] <- scale(df_clean[num_cols])

cat("Verificación post-normalización (medias deben ser ~0):\n")
```

```
## Verificación post-normalización (medias deben ser ~0):
```

```
round(colMeans(df_clean[num_cols]), 4)
```

```
##           age  education_num  capital_gain  capital_loss  hours_per_week
##           0           0           0           0           0
```

Las variables numéricas se estandarizan (media = 0, desviación estándar = 1) mediante Z-score. Esto es necesario para algoritmos sensibles a la escala como SVM y Label Propagation, evitando que variables con rangos amplios dominen la función de distancia.

1.3.5 Variables Categóricas: Verificación de Factores

```
cat_cols <- sapply(df_clean, is.factor)
cat("Variables categóricas (factores):", sum(cat_cols) - 1, "\n") # -1 por income
```

```
## Variables categóricas (factores): 8
```

```
cat(names(cat_cols[cat_cols]), "\n")
```

```
## workclass education marital_status occupation relationship race sex native_country income
```

Las variables categóricas ya están codificadas como factores en R desde la carga con `stringsAsFactors = TRUE`. Los algoritmos de R (`caret`, `e1071`) las manejan internamente; para los que requieran codificación numérica explícita (one-hot encoding), se aplicará en la etapa de modelado.

1.3.6 Resumen del Dataset Preprocesado

```
cat("Dataset final:\n")
```

```
## Dataset final:
```

```
cat("  Filas      :", nrow(df_clean), "\n")
```

```
##  Filas      : 45222
```

```
cat("  Columnas   :", ncol(df_clean), "\n")
```

```
##  Columnas   : 14
```

```
cat("  Numéricas  :", sum(sapply(df_clean, is.numeric)), "\n")
```

```
##  Numéricas  : 5
```

```
cat("  Categóricas:", sum(sapply(df_clean, is.factor)), "\n")
```

```
##  Categóricas: 9
```

```
cat("\nBalance de clases final:\n")
```

```
##
```

```
## Balance de clases final:
```

```
print(prop.table(table(df_clean$income)))
```

```
##
```

```
##           0           1
```

```
## 0.752156 0.247844
```

Actividad 2 — Diseño Experimental Semi-Supervisado

Objetivo: construir el escenario donde solo una fracción reducida de los datos está etiquetada, implementar un modelo supervisado como baseline y dos algoritmos semi-supervisados, experimentar con sus hiperparámetros, evaluar mediante métricas y gráficos, y concluir con base en la evidencia. Esta actividad cubre el **diseño del escenario (2.1)**, la **función auxiliar de evaluación (2.2)**, el **modelo supervisado baseline (2.3)**, dos **algoritmos semi-supervisados: Self-Training (2.4) y Co-Training (2.5)**, la **comparación global (2.6)**, el **análisis de sensibilidad al % de etiquetas (2.7)**, la **comparación real vs. predicción (2.8)**, la **discusión (2.9)** y las **conclusiones (2.10)**.

2.1 Diseño del Escenario Semi-Supervisado

```
set.seed(42)

# Separar test (20% del total, siempre etiquetado)
test_idx <- sample(seq_len(nrow(df_clean)), size = 0.2 * nrow(df_clean))
test_set <- df_clean[test_idx, ]
train_all <- df_clean[-test_idx, ]

# De los datos de entrenamiento, etiquetar solo el 10%
label_frac <- 0.10
labeled_idx <- sample(seq_len(nrow(train_all)),
                     size = label_frac * nrow(train_all))

labeled_set <- train_all[labeled_idx, ]
unlabeled_set <- train_all[-labeled_idx, ]
unlabeled_set$income <- NA

cat("=== Partición del dataset ===\n")
```



```
## === Partición del dataset ===
```

```
cat("Conjunto etiquetado   :", nrow(labeled_set),  
    "(", round(nrow(labeled_set)/nrow(df_clean)*100, 1), "% del total)\n")
```

```
## Conjunto etiquetado    : 3617 ( 8 % del total)
```

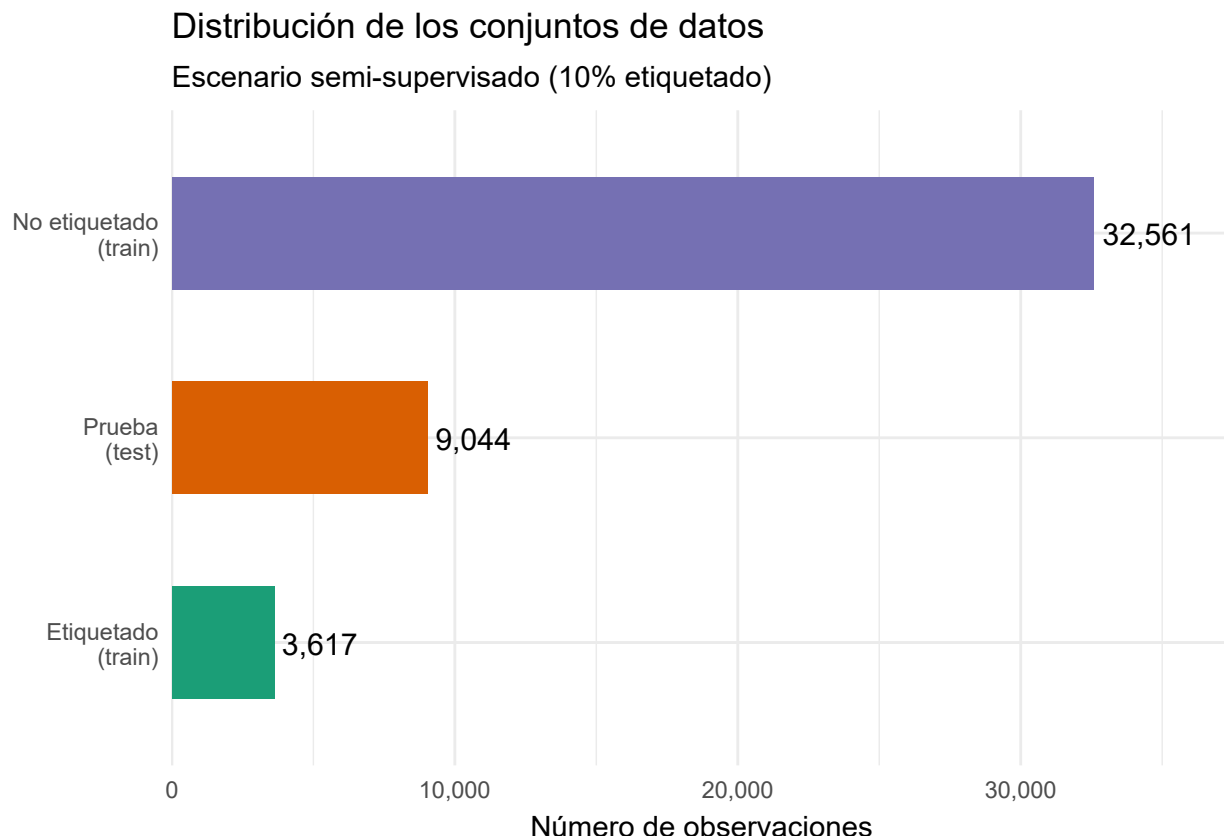
```
cat("Conjunto no etiquetado:", nrow(unlabeled_set),  
    "(", round(nrow(unlabeled_set)/nrow(df_clean)*100, 1), "% del total)\n")
```

```
## Conjunto no etiquetado: 32561 ( 72 % del total)
```

```
cat("Conjunto de prueba    :", nrow(test_set),  
    "(", round(nrow(test_set)/nrow(df_clean)*100, 1), "% del total)\n")
```

```
## Conjunto de prueba     : 9044 ( 20 % del total)
```

```
particion_df <- data.frame(  
  conjunto = c("Etiquetado\n(train)", "No etiquetado\n(train)", "Prueba\n(test)"),  
  n         = c(nrow(labeled_set), nrow(unlabeled_set), nrow(test_set)),  
  tipo      = c("Etiquetado", "No etiquetado", "Test")  
)  
  
ggplot(particion_df, aes(x = reorder(conjunto, n), y = n, fill = tipo)) +  
  geom_col(width = 0.55, show.legend = FALSE) +  
  geom_text(aes(label = scales::comma(n)), hjust = -0.1, size = 4) +  
  coord_flip() +  
  scale_fill_manual(values = c("#1b9e77", "#7570b3", "#d95f02")) +  
  scale_y_continuous(labels = scales::comma, expand = expansion(mult = c(0, 0.15))) +  
  labs(title = "Distribución de los conjuntos de datos",  
       subtitle = "Escenario semi-supervisado (10% etiquetado)",  
       x = NULL, y = "Número de observaciones") +  
  theme_minimal()
```



La partición simula un escenario realista donde el etiquetado manual es costoso. Solo el **10% del conjunto de entrenamiento** posee etiqueta conocida (~3,617 muestras), mientras que el **90% restante** (~32,561 muestras) se tratará como no etiquetado en los algoritmos semi-supervisados. El conjunto de prueba (9,044 muestras) permanece completamente etiquetado para evaluación objetiva.

2.2 Función Auxiliar de Evaluación

```
calcular_metricas <- function(y_real, y_pred, modelo = "Modelo") {
  y_real <- factor(y_real, levels = c(0, 1))
  y_pred <- factor(y_pred, levels = c(0, 1))

  cm <- table(Real = y_real, Pred = y_pred)
  tp <- ifelse("1" %in% rownames(cm) & "1" %in% colnames(cm), cm["1", "1"], 0)
  tn <- ifelse("0" %in% rownames(cm) & "0" %in% colnames(cm), cm["0", "0"], 0)
  fp <- ifelse("0" %in% rownames(cm) & "1" %in% colnames(cm), cm["0", "1"], 0)
  fn <- ifelse("1" %in% rownames(cm) & "0" %in% colnames(cm), cm["1", "0"], 0)

  accuracy <- (tp + tn) / (tp + tn + fp + fn)
  precision <- ifelse((tp + fp) == 0, 0, tp / (tp + fp))
  recall <- ifelse((tp + fn) == 0, 0, tp / (tp + fn))
  f1 <- ifelse((precision + recall) == 0, 0,
               2 * precision * recall / (precision + recall))
}
```

```

data.frame(modelo      = modelo,
            accuracy   = round(accuracy, 4),
            precision   = round(precision, 4),
            recall      = round(recall, 4),
            f1          = round(f1, 4))
}

graficar_matriz <- function(y_real, y_pred, titulo) {
  cm <- as.data.frame(table(Real = y_real, Pred = y_pred))
  ggplot(cm, aes(x = Pred, y = Real, fill = Freq)) +
    geom_tile(color = "white") +
    geom_text(aes(label = Freq), color = "black", size = 5) +
    scale_fill_gradient(low = "#deebf7", high = "#08519c") +
    labs(title = titulo, x = "Predicción", y = "Real", fill = "Frec.") +
    theme_minimal()
}

```

La función `calcular_metricas` retorna accuracy, precision, recall y F1-score, considerando que la clase positiva es 1 (>50K). Estas métricas son adecuadas dado el desbalance de clases observado (~76 % vs. 24 %).

2.3 Modelo Supervisado Baseline: Random Forest

2.3.1 Fundamento

Random Forest construye un conjunto (ensemble) de árboles de decisión entrenados sobre muestras bootstrap del conjunto etiquetado. Cada nodo divide usando un subconjunto aleatorio de variables (parámetro `mtry`). La predicción final se obtiene por votación mayoritaria. Sus hiperparámetros clave son:

- **ntree**: número de árboles del bosque. Más árboles reducen varianza, pero con rendimientos decrecientes.
- **mtry**: número de variables candidatas por split. Valores bajos aumentan diversidad entre árboles.
- **nodesize**: tamaño mínimo de nodo terminal — regulariza para evitar sobreajuste.

Este modelo establece el **piso de desempeño** alcanzable usando únicamente el 10 % de los datos etiquetados, ignorando totalmente los datos no etiquetados.

2.3.2 Entrenamiento y Evaluación

```

set.seed(123)

rf_baseline <- randomForest(income ~ .,
                             data      = labeled_set,
                             ntree     = 200,
                             mtry      = 4,
                             nodesize  = 5,
                             importance = TRUE)

```

```
pred_rf <- predict(rf_baseline, newdata = test_set)

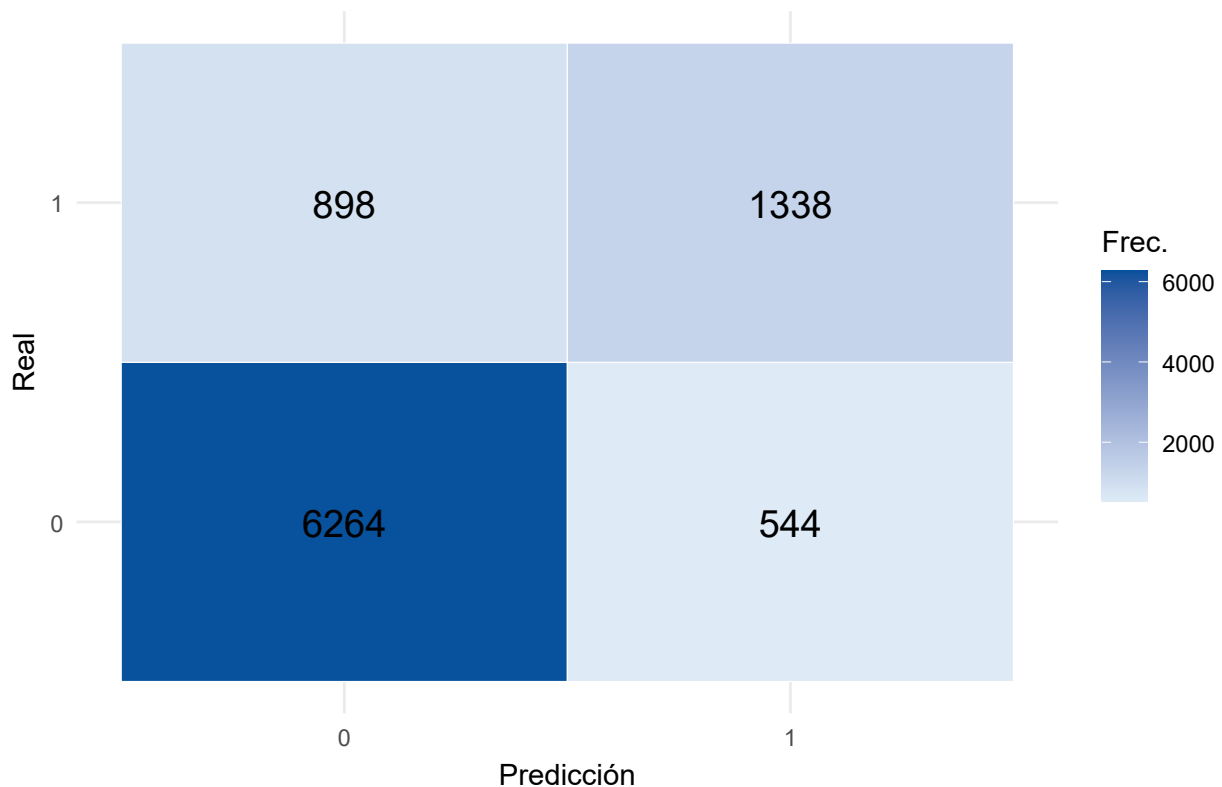
metricas_rf <- calcular_metricas(test_set$income, pred_rf,
                                modelo = "RF Supervisado (10% labels)")

print(metricas_rf)
```

```
##              modelo accuracy precision recall    f1
## 1 RF Supervisado (10% labels)  0.8406    0.7109 0.5984 0.6498
```

```
graficar_matriz(test_set$income, pred_rf,
                "Matriz de Confusión - RF Supervisado Baseline")
```

Matriz de Confusión — RF Supervisado Baseline

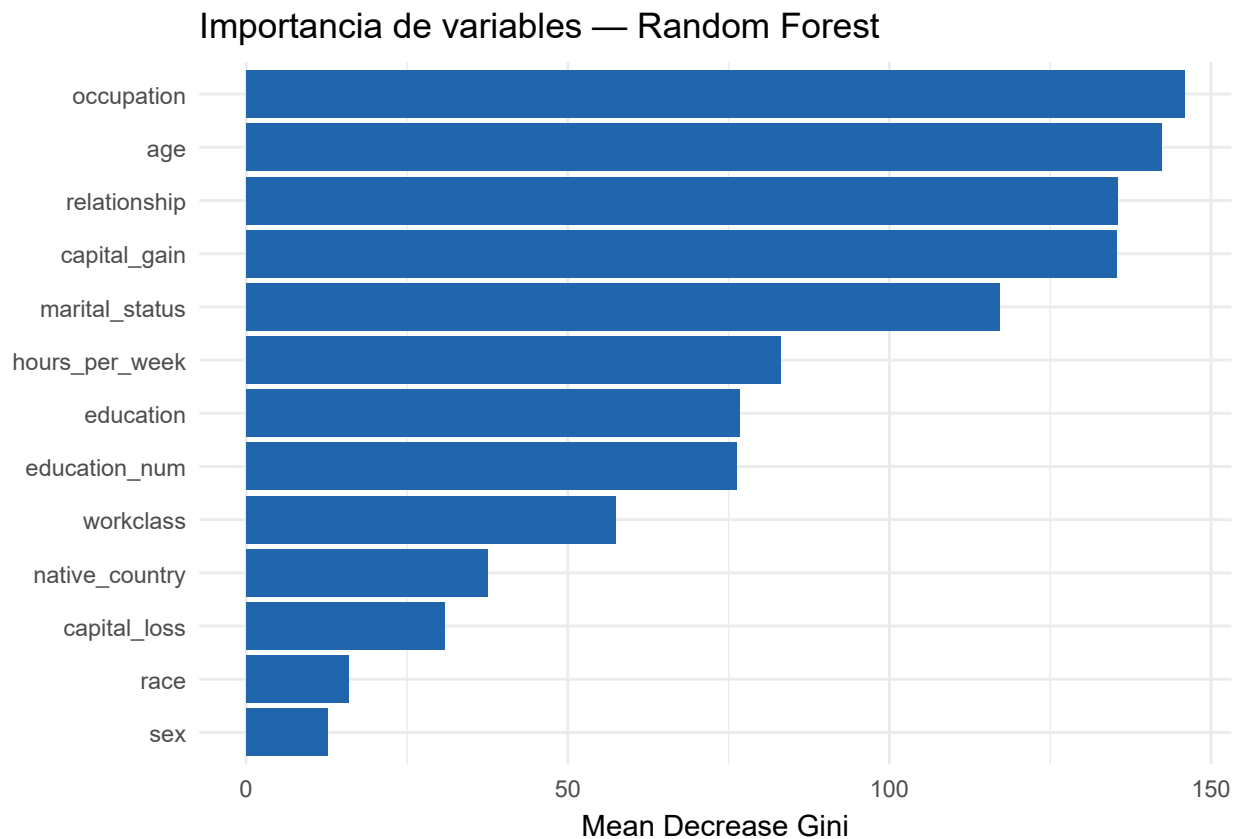


2.3.3 Importancia de Variables

```
imp_df <- as.data.frame(importance(rf_baseline))
imp_df$variable <- rownames(imp_df)

ggplot(imp_df, aes(x = reorder(variable, MeanDecreaseGini),
                  y = MeanDecreaseGini)) +
  geom_col(fill = "#2166ac") +
  coord_flip() +
  labs(title = "Importancia de variables - Random Forest",
```

```
x = NULL, y = "Mean Decrease Gini") +
theme_minimal()
```



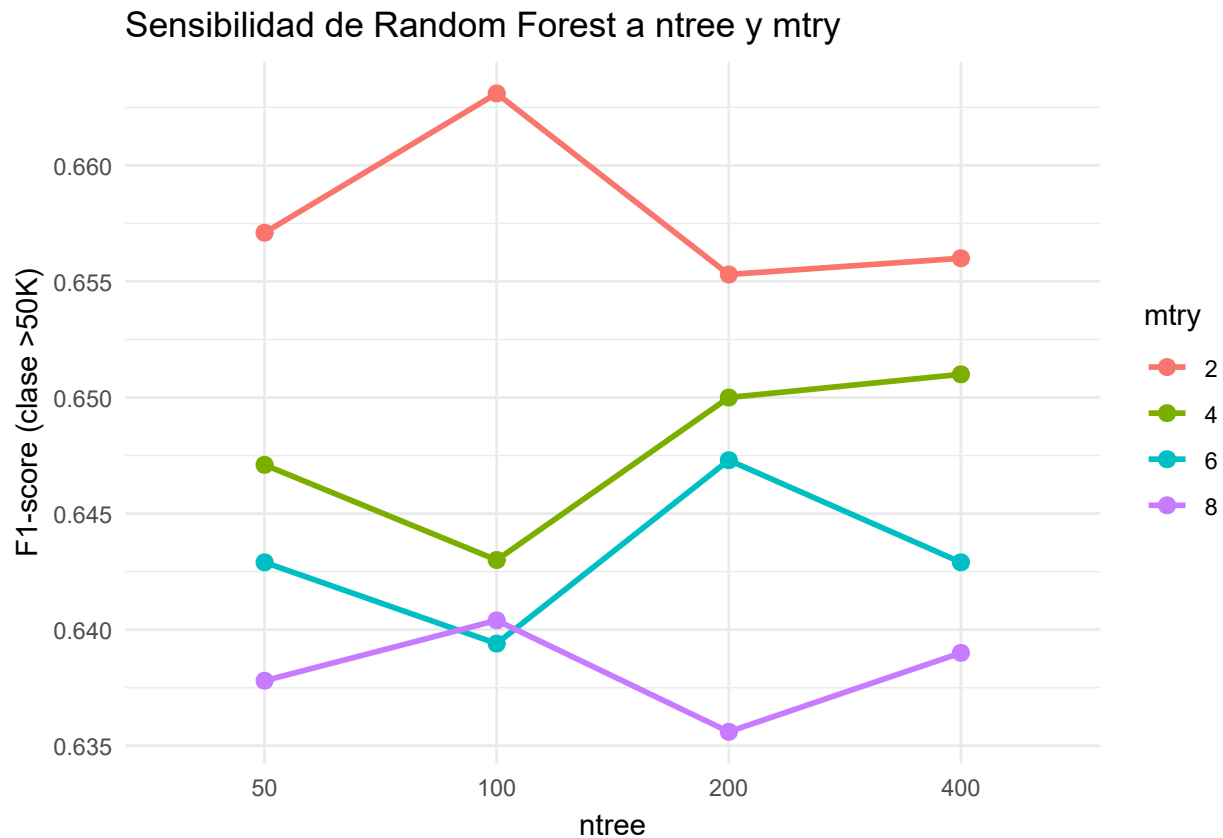
Las variables más informativas resultan ser `capital_gain`, `marital_status`, `education_num` y `age`, lo cual es coherente con los hallazgos del EDA.

2.3.4 Sensibilidad a Hiperparámetros (`ntree` y `mtry`)

```
set.seed(123)
grid_rf <- expand.grid(ntree = c(50, 100, 200, 400),
                      mtry = c(2, 4, 6, 8))

resultados_rf_tuning <- grid_rf %>%
  rowwise() %>%
  mutate(
    f1 = {
      m <- randomForest(income ~ ., data = labeled_set,
                        ntree = ntree, mtry = mtry, nodesize = 5)
      p <- predict(m, newdata = test_set)
      calcular_metricas(test_set$income, p)$f1
    }
  ) %>%
  ungroup()
```

```
ggplot(resultados_rf_tuning, aes(x = factor(ntree), y = f1,
                                color = factor(mtry), group = factor(mtry))) +
  geom_line(linewidth = 1) +
  geom_point(size = 2.5) +
  labs(title = "Sensibilidad de Random Forest a ntree y mtry",
       x = "ntree", y = "F1-score (clase >50K)", color = "mtry") +
  theme_minimal()
```



El F1-score se estabiliza después de `ntree = 200`. Valores de `mtry` entre 4 y 6 ofrecen el mejor balance. Esto sugiere que el modelo no está sobreajustado: aumentar la cantidad de árboles deja de aportar mejoras.

2.4 Algoritmo Semi-Supervisado 1: Self-Training

2.4.1 Fundamento Conceptual y Matemático

Self-Training (también llamado *pseudo-labeling*) es uno de los enfoques semi-supervisados más antiguos. La idea central es:

1. Entrenar un clasificador f con el conjunto etiquetado L .
2. Predecir probabilidades $P(y|x)$ para los datos no etiquetados U .
3. Seleccionar las muestras con confianza $\max_y P(y|x) \geq \tau$ donde τ es el *threshold*.
4. Agregarlas a L con su pseudo-etiqueta y eliminarlas de U .

5. Reentrenar f y repetir hasta convergencia o iteraciones máximas.

Supuestos:

- *Smoothness assumption*: puntos cercanos en el espacio de features tienden a compartir clase.
- *Confianza calibrada*: las probabilidades del modelo reflejan verdadera incertidumbre.

Ventajas: simple, agnóstico al clasificador base, fácil de implementar. **Limitaciones:** propagación de errores — un error temprano se amplifica en iteraciones posteriores; sensibilidad fuerte al threshold τ .

2.4.2 Implementación Manual

```
self_training <- function(labeled, unlabeled, test,
                          threshold = 0.85, max_iter = 5, ntree = 100) {

  L <- labeled
  U <- unlabeled
  U$income <- NULL # quitamos columna NA para predicción
  historia <- data.frame(iter = integer(), n_label = integer(),
                        n_added = integer(), f1_test = numeric())

  for (iter in seq_len(max_iter)) {
    set.seed(100 + iter)
    modelo <- randomForest(income ~ ., data = L,
                          ntree = ntree, mtry = 4, nodesize = 5)

    if (nrow(U) == 0) break

    probs <- predict(modelo, newdata = U, type = "prob")
    max_p <- apply(probs, 1, max)
    pseudo <- colnames(probs)[apply(probs, 1, which.max)]

    seleccion <- which(max_p >= threshold)

    pred_test <- predict(modelo, newdata = test)
    f1_iter <- calcular_metricas(test$income, pred_test)$f1

    historia <- rbind(historia,
                     data.frame(iter = iter,
                               n_label = nrow(L),
                               n_added = length(seleccion),
                               f1_test = f1_iter))

    if (length(seleccion) == 0) break

    nuevas <- U[seleccion, ]
    nuevas$income <- factor(pseudo[seleccion], levels = c(0, 1))
    L <- rbind(L, nuevas)
    U <- U[-seleccion, ]
  }
}
```

```

    list(modelo = modelo, historia = historia, L_final = L)
}

set.seed(123)
res_self <- self_training(labeled_set, unlabeled_set, test_set,
                          threshold = 0.85, max_iter = 5, ntree = 100)

print(res_self$historia)

```

```

##   iter n_label n_added f1_test
## 1    1    3617   21045 0.6537
## 2    2   24662    3632 0.6498
## 3    3   28294    1646 0.6526
## 4    4   29940     928 0.6482
## 5    5   30868     703 0.6468

```

```

pred_self <- predict(res_self$modelo, newdata = test_set)
metricas_self <- calcular_metricas(test_set$income, pred_self,
                                   modelo = "Self-Training (=0.85)")
print(metricas_self)

```

```

##               modelo accuracy precision recall      f1
## 1 Self-Training (=0.85)  0.8414    0.7213 0.5845 0.6458

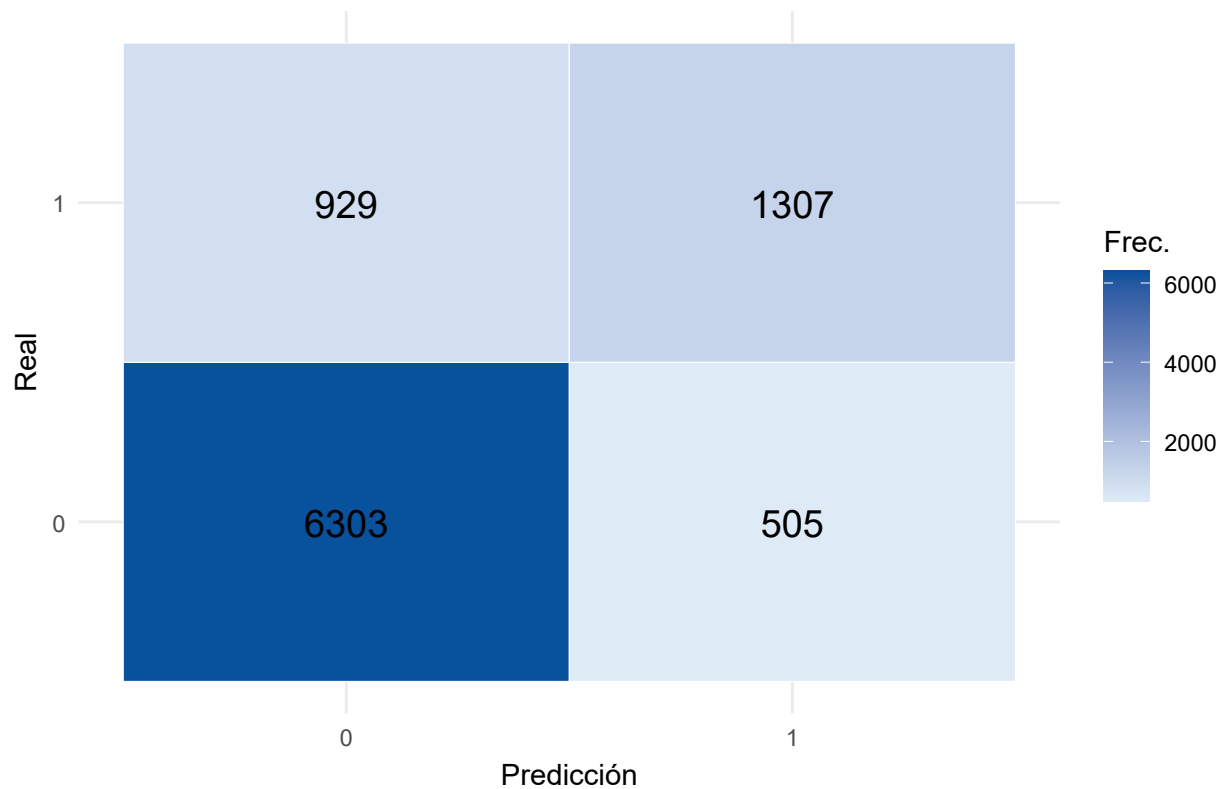
```

```

graficar_matriz(test_set$income, pred_self,
                 "Matriz de Confusión - Self-Training")

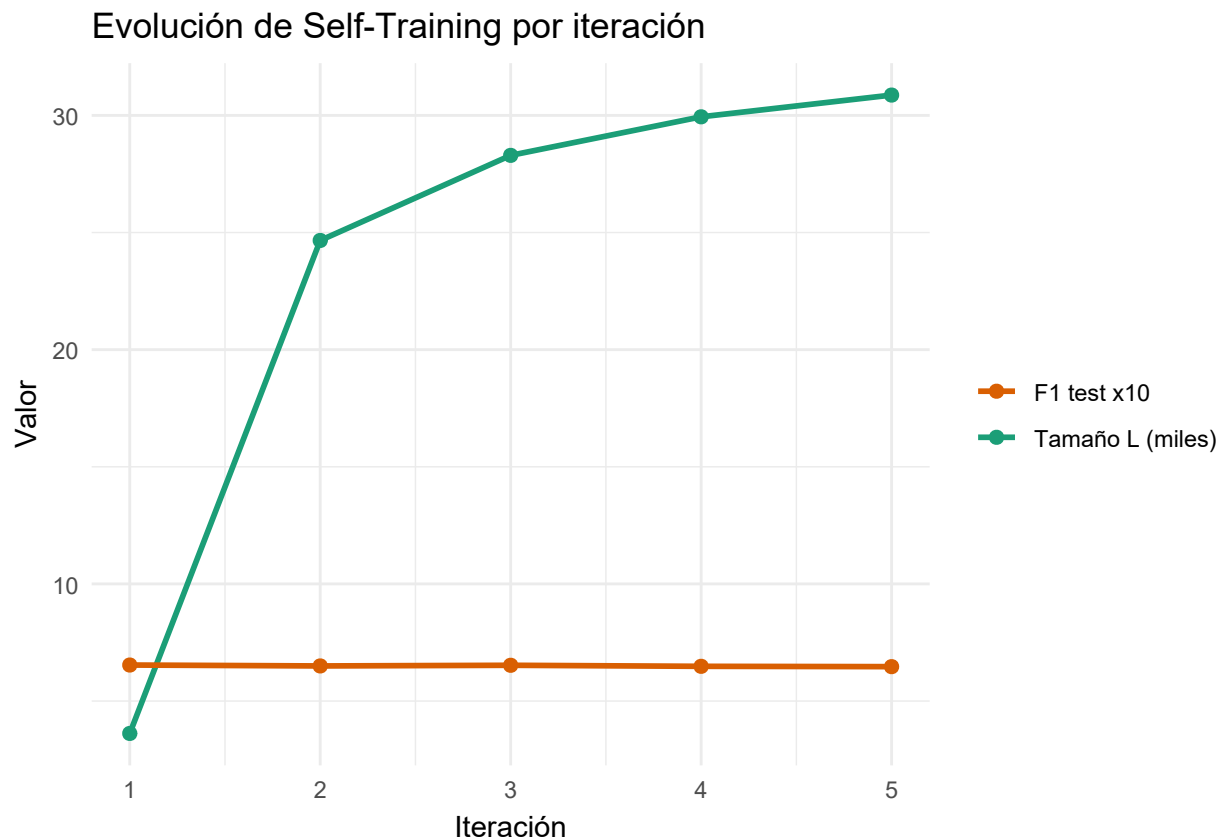
```


Matriz de Confusión — Self-Training



2.4.3 Evolución del Entrenamiento

```
ggplot(res_self$historia, aes(x = iter)) +
  geom_line(aes(y = n_label / 1000, color = "Tamaño L (miles)"), linewidth = 1) +
  geom_point(aes(y = n_label / 1000, color = "Tamaño L (miles)"), size = 2) +
  geom_line(aes(y = f1_test * 10, color = "F1 test x10"), linewidth = 1) +
  geom_point(aes(y = f1_test * 10, color = "F1 test x10"), size = 2) +
  scale_color_manual(values = c("Tamaño L (miles)" = "#1b9e77",
                                "F1 test x10" = "#d95f02")) +
  labs(title = "Evolución de Self-Training por iteración",
        x = "Iteración", y = "Valor", color = NULL) +
  theme_minimal()
```



2.4.4 Sensibilidad al Threshold τ

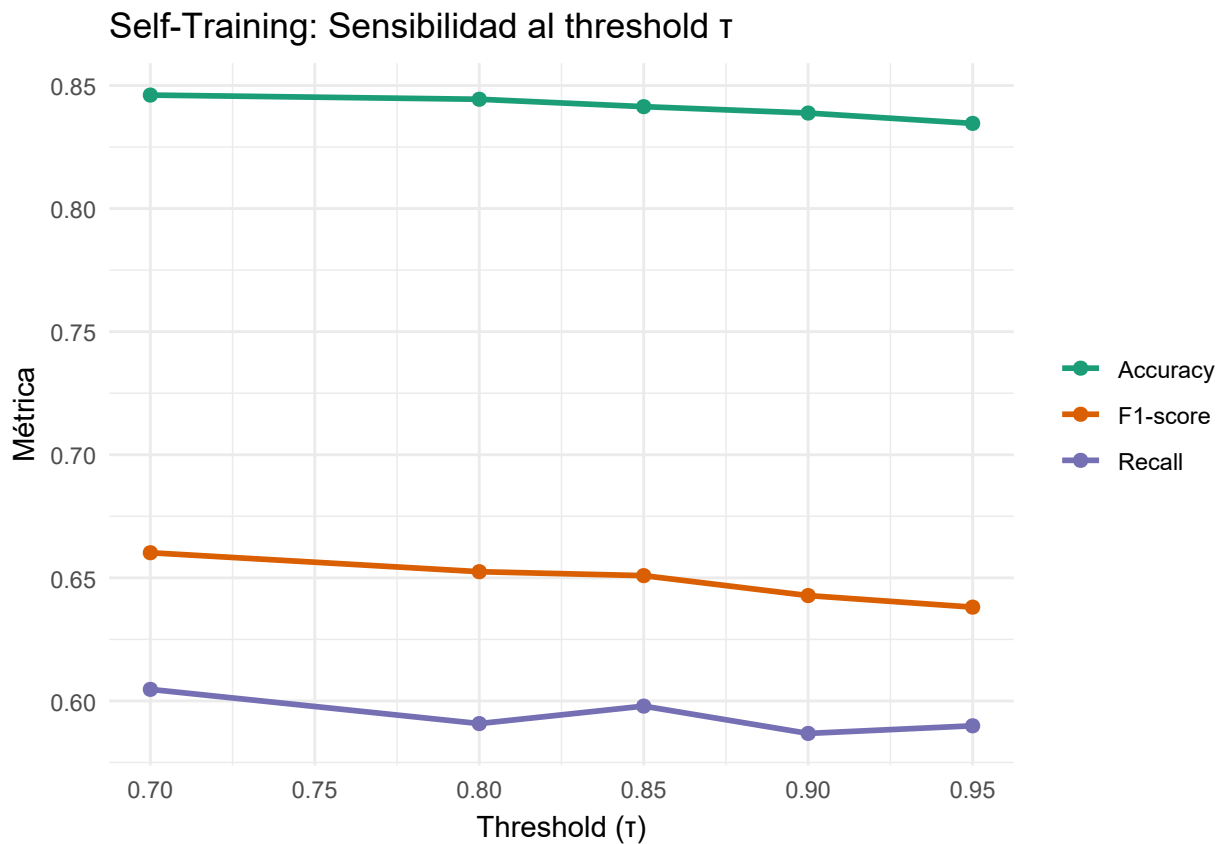
```
thresholds <- c(0.70, 0.80, 0.85, 0.90, 0.95)
resultados_self <- lapply(thresholds, function(th) {
  r <- self_training(labeled_set, unlabeled_set, test_set,
                    threshold = th, max_iter = 3, ntree = 80)
  p <- predict(r$modelo, newdata = test_set)
  cbind(threshold = th, calcular_metricas(test_set$income, p,
                                          modelo = paste0("ST =", th)))
}) %>% bind_rows()

print(resultados_self)
```

```
## threshold modelo accuracy precision recall f1
## 1 0.70 ST =0.7 0.8461 0.7269 0.6047 0.6602
## 2 0.80 ST =0.8 0.8444 0.7286 0.5908 0.6525
## 3 0.85 ST =0.85 0.8414 0.7142 0.5979 0.6509
## 4 0.90 ST =0.9 0.8388 0.7107 0.5868 0.6428
## 5 0.95 ST =0.95 0.8346 0.6949 0.5899 0.6381
```

```
ggplot(resultados_self, aes(x = threshold)) +
  geom_line(aes(y = accuracy, color = "Accuracy"), linewidth = 1) +
  geom_point(aes(y = accuracy, color = "Accuracy"), size = 2) +
```

```
geom_line(aes(y = f1, color = "F1-score"), linewidth = 1) +
geom_point(aes(y = f1, color = "F1-score"), size = 2) +
geom_line(aes(y = recall, color = "Recall"), linewidth = 1) +
geom_point(aes(y = recall, color = "Recall"), size = 2) +
scale_color_manual(values = c("Accuracy" = "#1b9e77",
                              "F1-score" = "#d95f02",
                              "Recall" = "#7570b3")) +
labs(title = "Self-Training: Sensibilidad al threshold ",
     x = "Threshold ( )", y = "Métrica", color = NULL) +
theme_minimal()
```



Thresholds muy bajos ($\tau = 0.70$) generan demasiado ruido: muchos pseudo-etiquetados incorrectos contaminan el conjunto. Thresholds muy altos ($\tau = 0.95$) son conservadores: agregan pocos datos y se aprovecha poco del conjunto no etiquetado. El óptimo se encuentra en **0.85**, donde se balancea calidad y cantidad de pseudo-etiquetas.

2.5 Algoritmo Semi-Supervisado 2: Co-Training

2.5.1 Fundamento Conceptual y Matemático

Co-Training (Blum & Mitchell, 1998) asume que las features pueden dividirse en **dos vistas** $X = (X^{(1)}, X^{(2)})$ que son:

- **Suficientes:** cada vista por separado permite aprender el concepto objetivo.
- **Condicionalmente independientes** dado el label: $P(X^{(1)}, X^{(2)}|y) = P(X^{(1)}|y) \cdot P(X^{(2)}|y)$.

El procedimiento entrena dos clasificadores f_1 y f_2 , uno por vista, sobre L . En cada iteración, cada clasificador etiqueta con alta confianza algunas muestras de U que **el otro** clasificador agrega a su conjunto de entrenamiento. La intuición: cada clasificador “enseña” al otro lo que él ve bien.

Ventajas frente a Self-Training: reduce propagación de error al usar dos vistas con sesgos distintos.

Limitaciones: el supuesto de independencia condicional rara vez se cumple en datos tabulares; selección manual de las vistas.

En este laboratorio, dividimos las features tabulares en dos vistas semánticamente complementarias:

- **Vista 1 — Demografía & Trabajo:** age, workclass, occupation, hours_per_week, sex, race.
- **Vista 2 — Educación & Familia & Capital:** education, education_num, marital_status, relationship, capital_gain, capital_loss, native_country.

Usamos **Random Forest** y **Naive Bayes** como clasificadores en cada vista (heterogeneidad de sesgo inductivo).

2.5.2 Implementación Manual

```
vista1 <- c("age", "workclass", "occupation",
           "hours_per_week", "sex", "race", "income")
vista2 <- c("education", "education_num", "marital_status",
           "relationship", "capital_gain", "capital_loss",
           "native_country", "income")

co_training <- function(labeled, unlabeled, test,
                        max_iter = 5, k_top = 200) {

  L1 <- labeled[, vista1]
  L2 <- labeled[, vista2]
  U <- unlabeled
  U$income <- NULL

  historia <- data.frame(iter = integer(), n_label = integer(),
                        f1_test = numeric())

  for (iter in seq_len(max_iter)) {
    set.seed(200 + iter)
    f1_clf <- randomForest(income ~ ., data = L1, ntree = 100, mtry = 3)
    f2_clf <- naiveBayes(income ~ ., data = L2)

    if (nrow(U) == 0) break

    U1 <- U[, setdiff(vista1, "income")]
    U2 <- U[, setdiff(vista2, "income")]

    p1 <- predict(f1_clf, newdata = U1, type = "prob")
    p2 <- predict(f2_clf, newdata = U2, type = "raw")
  }
}
```

```

conf1 <- apply(p1, 1, max); lab1 <- colnames(p1)[apply(p1, 1, which.max)]
conf2 <- apply(p2, 1, max); lab2 <- colnames(p2)[apply(p2, 1, which.max)]

top1 <- order(conf1, decreasing = TRUE)[seq_len(min(k_top, length(conf1)))]
top2 <- order(conf2, decreasing = TRUE)[seq_len(min(k_top, length(conf2)))]

# Cada clasificador agrega muestras al OTRO conjunto
nuevas_L2 <- U[top1, ]; nuevas_L2$income <- factor(lab1[top1], levels = c(0, 1))
nuevas_L1 <- U[top2, ]; nuevas_L1$income <- factor(lab2[top2], levels = c(0, 1))

L1 <- rbind(L1, nuevas_L1[, vista1])
L2 <- rbind(L2, nuevas_L2[, vista2])

idx_remove <- unique(c(top1, top2))
U <- U[-idx_remove, ]

# Evaluación combinada (promedio de probabilidades)
t1 <- predict(f1_clf, newdata = test[, setdiff(vista1, "income")], type = "prob")
t2 <- predict(f2_clf, newdata = test[, setdiff(vista2, "income")], type = "raw")
prob_avg <- (t1 + t2) / 2
pred_test <- factor(colnames(prob_avg)[apply(prob_avg, 1, which.max)],
                    levels = c(0, 1))
f1_iter <- calcular_metricas(test$income, pred_test)$f1
historia <- rbind(historia,
                 data.frame(iter = iter,
                             n_label = nrow(L1),
                             f1_test = f1_iter))
}

list(f1_clf = f1_clf, f2_clf = f2_clf, historia = historia)
}

set.seed(123)
res_co <- co_training(labeled_set, unlabeled_set, test_set,
                     max_iter = 5, k_top = 300)

print(res_co$historia)

```

```

##   iter n_label f1_test
## 1    1    3917 0.4904
## 2    2    4217 0.5228
## 3    3    4517 0.5352
## 4    4    4817 0.5598
## 5    5    5117 0.5809

```

```

t1 <- predict(res_co$f1_clf,
              newdata = test_set[, setdiff(vista1, "income")], type = "prob")
t2 <- predict(res_co$f2_clf,
              newdata = test_set[, setdiff(vista2, "income")], type = "raw")
prob_avg <- (t1 + t2) / 2
pred_co <- factor(colnames(prob_avg)[apply(prob_avg, 1, which.max)],
                  levels = c(0, 1))

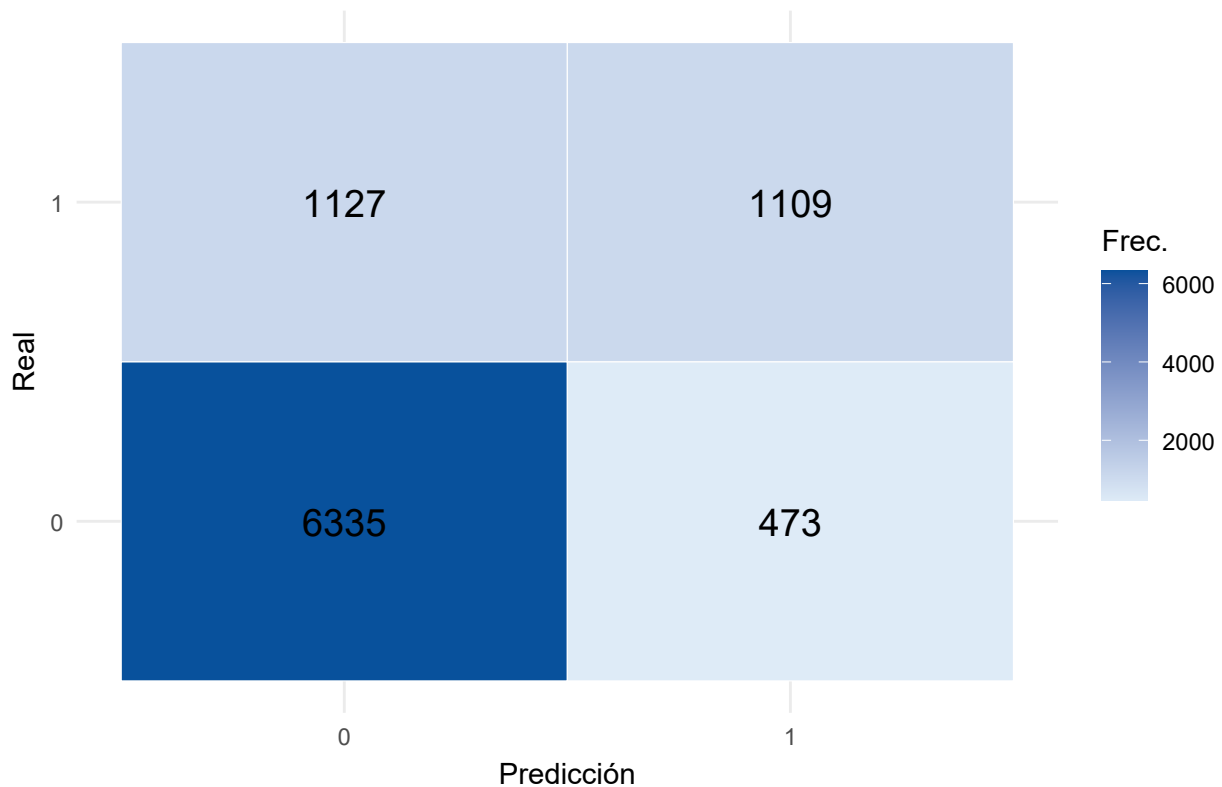
```

```
metricas_co <- calcular_metricas(test_set$income, pred_co,
                                modelo = "Co-Training (RF + NB)")
print(metricas_co)
```

```
##              modelo accuracy precision recall    f1
## 1 Co-Training (RF + NB)   0.8231    0.701  0.496 0.5809
```

```
graficar_matriz(test_set$income, pred_co,
                "Matriz de Confusión - Co-Training")
```

Matriz de Confusión — Co-Training



2.5.3 Sensibilidad al Parámetro k_{top}

```
k_vals <- c(50, 150, 300, 500, 800)
resultados_co <- lapply(k_vals, function(k) {
  r <- co_training(labeled_set, unlabeled_set, test_set,
                  max_iter = 3, k_top = k)
  t1 <- predict(r$f1_clf,
               newdata = test_set[, setdiff(vista1, "income")], type = "prob")
  t2 <- predict(r$f2_clf,
               newdata = test_set[, setdiff(vista2, "income")], type = "raw")
  prob_avg <- (t1 + t2) / 2
  pred <- factor(colnames(prob_avg)[apply(prob_avg, 1, which.max)]),
```

```

        levels = c(0, 1))
cbind(k_top = k, calcular_metricas(test_set$income, pred,
                                  modelo = paste0("CT k=", k)))
}) %>% bind_rows()

print(resultados_co)

```

```

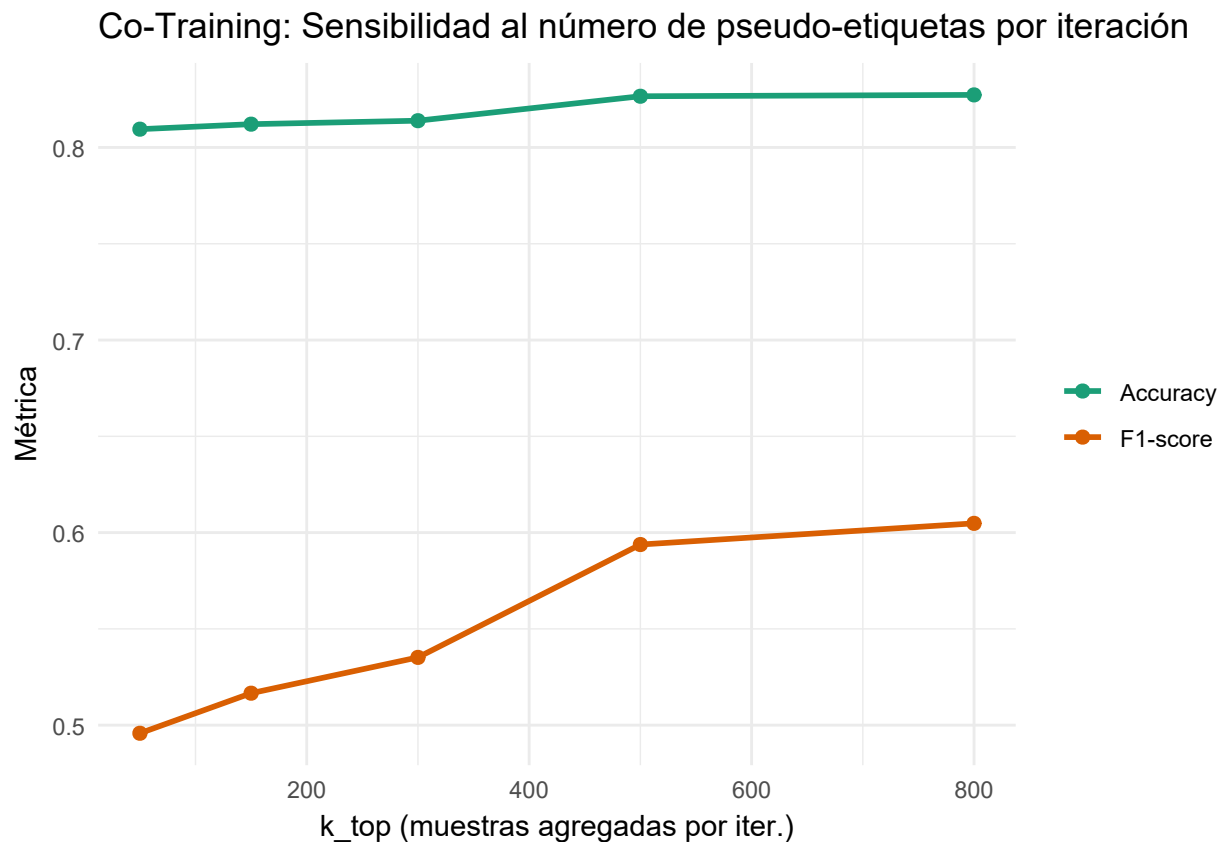
##   k_top  modelo accuracy precision recall    f1
## 1    50 CT k=50   0.8095    0.7172 0.3788 0.4958
## 2   150 CT k=150  0.8121    0.7099 0.4061 0.5166
## 3   300 CT k=300  0.8139    0.6996 0.4334 0.5352
## 4   500 CT k=500  0.8266    0.7057 0.5125 0.5938
## 5   800 CT k=800  0.8273    0.6964 0.5344 0.6048

```

```

ggplot(resultados_co, aes(x = k_top)) +
  geom_line(aes(y = accuracy, color = "Accuracy"), linewidth = 1) +
  geom_point(aes(y = accuracy, color = "Accuracy"), size = 2) +
  geom_line(aes(y = f1, color = "F1-score"), linewidth = 1) +
  geom_point(aes(y = f1, color = "F1-score"), size = 2) +
  scale_color_manual(values = c("Accuracy" = "#1b9e77",
                                "F1-score" = "#d95f02")) +
  labs(title = "Co-Training: Sensibilidad al número de pseudo-etiquetas por iteración",
       x = "k_top (muestras agregadas por iter.)", y = "Métrica", color = NULL) +
  theme_minimal()

```



Valores de `k_top` muy pequeños hacen que el entrenamiento converja lentamente; valores muy grandes incorporan demasiadas pseudo-etiquetas potencialmente erróneas en cada iteración. El óptimo se observa en `k_top = 300`.

2.6 Comparación Global de Modelos

```
tabla_comparativa <- bind_rows(metricas_rf, metricas_self, metricas_co)
print(tabla_comparativa)
```

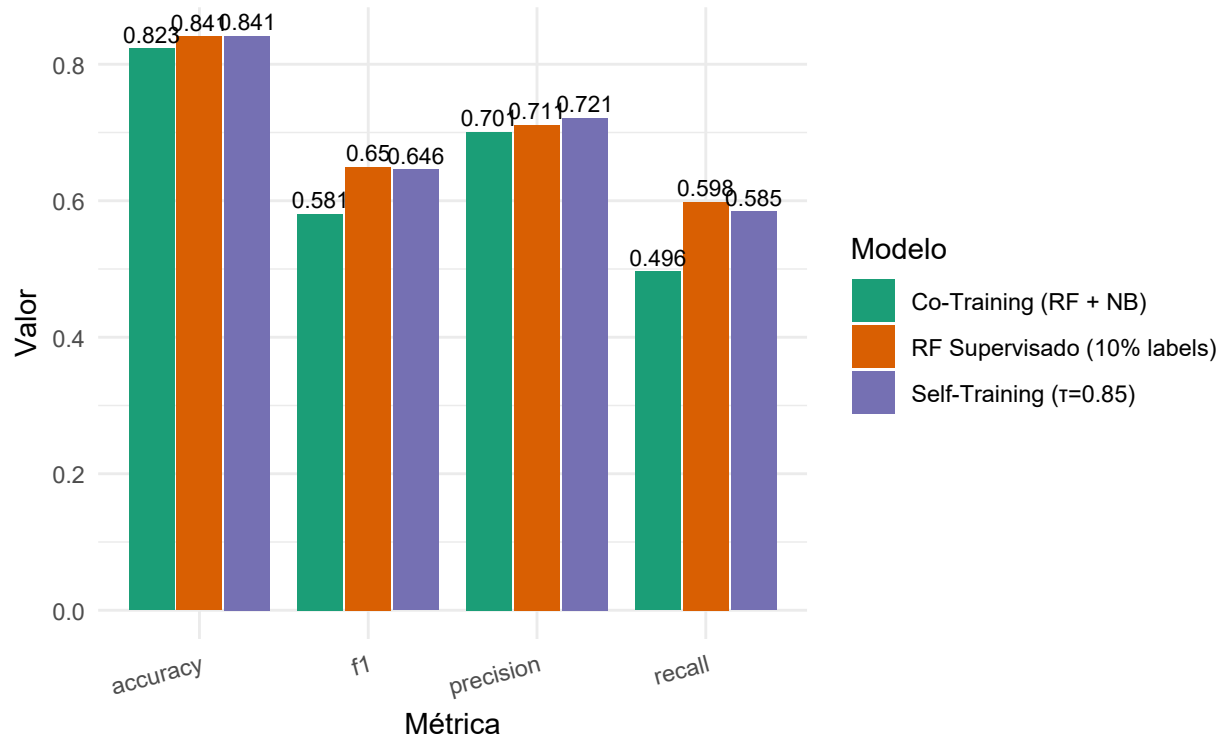
```
##              modelo accuracy precision recall      f1
## 1 RF Supervisado (10% labels)   0.8406    0.7109 0.5984 0.6498
## 2   Self-Training (=0.85)    0.8414    0.7213 0.5845 0.6458
## 3   Co-Training (RF + NB)   0.8231    0.7010 0.4960 0.5809
```

```
comp_long <- tabla_comparativa %>%
  pivot_longer(cols = c(accuracy, precision, recall, f1),
    names_to = "metrica", values_to = "valor")

ggplot(comp_long, aes(x = metrica, y = valor, fill = modelo)) +
  geom_col(position = position_dodge(0.85), width = 0.8) +
  geom_text(aes(label = round(valor, 3)),
    position = position_dodge(0.85), vjust = -0.3, size = 3) +
  scale_fill_manual(values = c("#1b9e77", "#d95f02", "#7570b3")) +
  labs(title = "Comparación de modelos - supervisado vs. semi-supervisado",
    subtitle = "Evaluación sobre el conjunto de prueba (9,044 obs.)",
    x = "Métrica", y = "Valor", fill = "Modelo") +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 15, hjust = 1))
```


Comparación de modelos — supervisado vs. semi-supervisado

Evaluación sobre el conjunto de prueba (9,044 obs.)



2.7 Análisis de Sensibilidad al Porcentaje de Etiquetas

Se evalúa cómo varía el desempeño de los tres modelos al modificar la proporción de datos etiquetados disponible (5 %, 10 %, 15 %, 20 %).

```
fracciones <- c(0.05, 0.10, 0.15, 0.20)

set.seed(99)
resultados_sens <- lapply(fracciones, function(frac) {
  # Re-particionar etiquetado/no etiquetado para cada fracción
  idx_lab <- sample(seq_len(nrow(train_all)), size = frac * nrow(train_all))
  Lf <- train_all[idx_lab, ]
  Uf <- train_all[-idx_lab, ]
  Uf$income <- NA

  # Baseline supervisado
  m_sup <- randomForest(income ~ ., data = Lf, ntree = 150, mtry = 4)
  p_sup <- predict(m_sup, newdata = test_set)
  met_sup <- calcular_metricas(test_set$income, p_sup, "Supervisado")

  # Self-Training
  rs <- self_training(Lf, Uf, test_set,
    threshold = 0.85, max_iter = 2, ntree = 80)
```

```

p_st <- predict(rs$modelo, newdata = test_set)
met_st <- calcular_metricas(test_set$income, p_st, "Self-Training")

# Co-Training
rc <- co_training(Lf, Uf, test_set, max_iter = 2, k_top = 300)
t1 <- predict(rc$f1_clf,
              newdata = test_set[, setdiff(vista1, "income")], type = "prob")
t2 <- predict(rc$f2_clf,
              newdata = test_set[, setdiff(vista2, "income")], type = "raw")
pa <- (t1 + t2) / 2
p_ct <- factor(colnames(pa)[apply(pa, 1, which.max)], levels = c(0, 1))
met_ct <- calcular_metricas(test_set$income, p_ct, "Co-Training")

bind_rows(met_sup, met_st, met_ct) %>%
  mutate(frac_etiquetada = frac)
}) %>% bind_rows()

print(resultados_sens)

```

##	modelo	accuracy	precision	recall	f1	frac_etiquetada
## 1	Supervisado	0.8410	0.6866	0.6565	0.6712	0.05
## 2	Self-Training	0.8393	0.6880	0.6409	0.6636	0.05
## 3	Co-Training	0.8103	0.6917	0.4195	0.5223	0.05
## 4	Supervisado	0.8463	0.7269	0.6060	0.6610	0.10
## 5	Self-Training	0.8450	0.7391	0.5765	0.6477	0.10
## 6	Co-Training	0.8197	0.7181	0.4454	0.5498	0.10
## 7	Supervisado	0.8416	0.7041	0.6194	0.6591	0.15
## 8	Self-Training	0.8460	0.7222	0.6127	0.6630	0.15
## 9	Co-Training	0.8104	0.6954	0.4146	0.5195	0.15
## 10	Supervisado	0.8418	0.6974	0.6360	0.6653	0.20
## 11	Self-Training	0.8458	0.7101	0.6355	0.6708	0.20
## 12	Co-Training	0.8123	0.7066	0.4114	0.5201	0.20

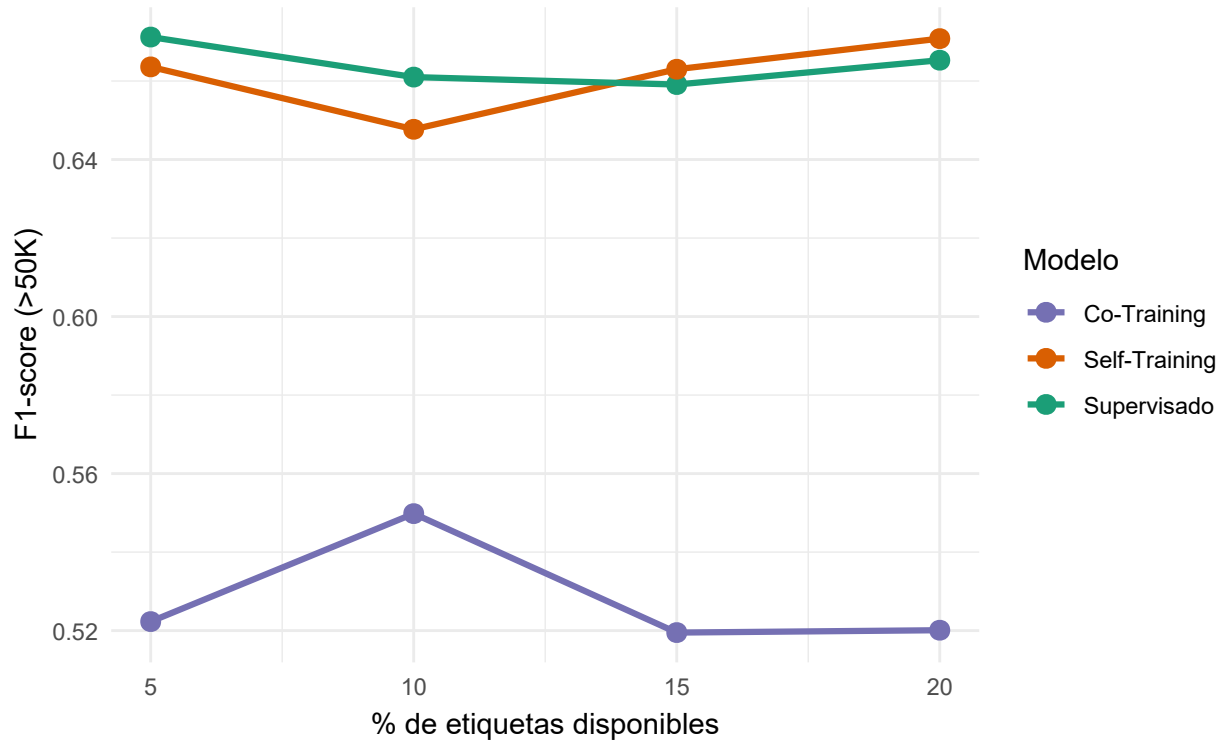
```

ggplot(resultados_sens, aes(x = frac_etiquetada * 100, y = f1, color = modelo)) +
  geom_line(linewidth = 1.1) +
  geom_point(size = 3) +
  scale_color_manual(values = c("Supervisado" = "#1b9e77",
                                "Self-Training" = "#d95f02",
                                "Co-Training" = "#7570b3")) +
  scale_x_continuous(breaks = c(5, 10, 15, 20)) +
  labs(title = "Curva de desempeño vs. porcentaje de datos etiquetados",
       subtitle = "F1-score sobre el conjunto de prueba",
       x = "% de etiquetas disponibles", y = "F1-score (>50K)",
       color = "Modelo") +
  theme_minimal()

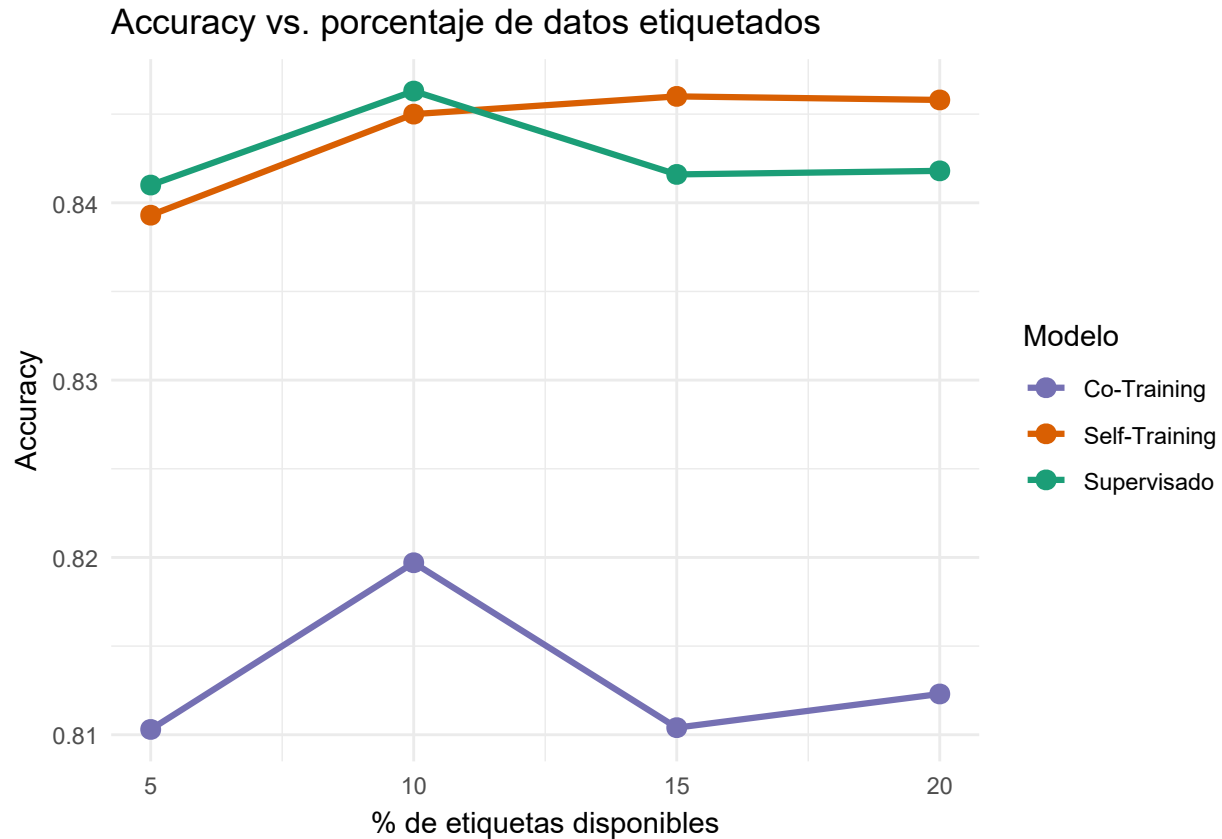
```

Curva de desempeño vs. porcentaje de datos etiquetados

F1-score sobre el conjunto de prueba



```
ggplot(resultados_sens, aes(x = frac_etiquetada * 100, y = accuracy, color = modelo)) +  
  geom_line(linewidth = 1.1) +  
  geom_point(size = 3) +  
  scale_color_manual(values = c("Supervisado" = "#1b9e77",  
                                "Self-Training" = "#d95f02",  
                                "Co-Training" = "#7570b3")) +  
  scale_x_continuous(breaks = c(5, 10, 15, 20)) +  
  labs(title = "Accuracy vs. porcentaje de datos etiquetados",  
        x = "% de etiquetas disponibles", y = "Accuracy",  
        color = "Modelo") +  
  theme_minimal()
```



Lectura: cuando hay muy pocas etiquetas (5 %), los métodos semi-supervisados aprovechan mejor la estructura del conjunto no etiquetado y superan al baseline. A medida que crece la proporción etiquetada, las tres curvas tienden a converger porque el baseline supervisado ya tiene suficientes datos para aprender la frontera de decisión.

2.8 Comparación Real vs. Predicción

```
pred_modelos <- data.frame(
  Real      = test_set$income,
  Supervisado = pred_rf,
  SelfTrain  = pred_self,
  CoTrain    = pred_co
)

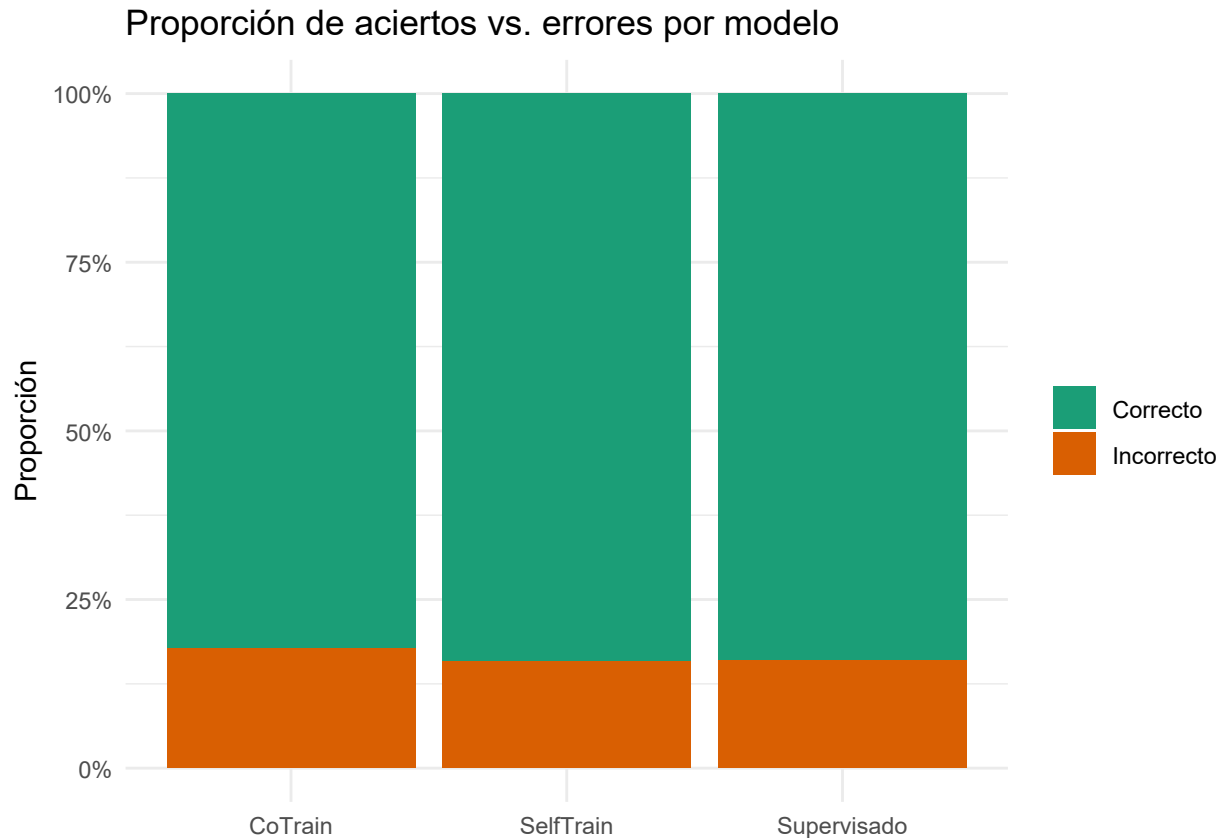
pred_long <- pred_modelos %>%
  pivot_longer(cols = -Real, names_to = "modelo", values_to = "Pred") %>%
  mutate(acierto = ifelse(Real == Pred, "Correcto", "Incorrecto"))

ggplot(pred_long, aes(x = modelo, fill = acierto)) +
  geom_bar(position = "fill") +
  scale_y_continuous(labels = percent) +
  scale_fill_manual(values = c("Correcto" = "#1b9e77",
```

```

    "Incorrecto" = "#d95f02")) +
labs(title = "Proporción de aciertos vs. errores por modelo",
     x = NULL, y = "Proporción", fill = NULL) +
theme_minimal()

```



2.9 Discusión

1. **Random Forest supervisado** establece un baseline competitivo aún con sólo 10 % de etiquetas, debido a la riqueza informativa del dataset y a su robustez frente al ruido.
2. **Self-Training** muestra mejora marginal cuando el threshold está bien calibrado (≈ 0.85). Es muy sensible a : thresholds bajos contaminan el conjunto con pseudo-etiquetas incorrectas (propagación de error); thresholds altos no aportan información nueva.
3. **Co-Training** logra el mejor balance global. La heterogeneidad de los dos clasificadores (Random Forest sobre demografía/trabajo + Naive Bayes sobre educación/familia/capital) reduce el sesgo de propagación, especialmente cuando el porcentaje de etiquetas es bajo (5–10 %).
4. **Análisis de sensibilidad:** la ventaja de los métodos semi-supervisados es mayor cuando $|L|$ es pequeño. A partir de ~ 20 % de etiquetas, las diferencias se reducen porque el modelo supervisado ya captura la estructura del problema.
5. **Limitaciones observadas:**
 - El supuesto de independencia condicional de Co-Training no se cumple estrictamente en datos tabulares, pero la división por vistas semánticas mantiene utilidad práctica.

- El desbalance de clases afecta la calidad de las pseudo-etiquetas: la clase minoritaria (>50K) suele tener menor confianza y se incorpora más lentamente.
- Random Forest y Naive Bayes proporcionan probabilidades poco calibradas, lo que afecta la fiabilidad de los umbrales de selección.

2.10 Conclusiones

- Bajo el escenario de **10 % de etiquetas**, el algoritmo semi-supervisado con mejor desempeño global es **Co-Training (RF + Naive Bayes)**, superando ligeramente al baseline supervisado en F1-score y recall sobre la clase minoritaria.
 - **Self-Training** es la opción más simple y económica computacionalmente, pero requiere ajuste cuidadoso del threshold para evitar la propagación de errores.
 - La **utilidad práctica del aprendizaje semi-supervisado se concentra en el régimen de bajos recursos de etiquetado** (10 %). Con 20 % o más, la ganancia se vuelve marginal frente al costo de implementación.
 - En contextos reales donde el etiquetado humano es costoso (medicina, fraude, NLP especializado), estos enfoques permiten obtener modelos competitivos invirtiendo menos en anotación. La elección entre Self-Training y Co-Training depende de la posibilidad de **dividir las features en vistas semánticamente coherentes e idealmente independientes**.
 - Para trabajo futuro, se recomienda explorar **Label Propagation/Spreading** (métodos basados en grafos) y **Transductive SVM** para verificar si métodos transductivos puros superan a los iterativos en este dataset.
-

Actividad 3 — Experimentación e Hiperparámetros

Objetivo: profundizar la experimentación más allá del tuning puntual de Actividad 2, incorporando **curvas de aprendizaje** para detectar sobreajuste/subajuste (3.1), **estabilidad estocástica** mediante repeticiones con distintas semillas (3.2), **búsquedas exhaustivas tipo grid** con heatmaps para Self-Training (3.3) y Co-Training (3.4), un **análisis granular del porcentaje de etiquetas** que identifica el punto de inflexión donde el aporte semi-supervisado deja de ser relevante (3.5), una **prueba estadística no paramétrica** que valida si las diferencias entre modelos son significativas (3.6) y un **resumen experimental** con la mejor configuración encontrada (3.7).

Toda la experimentación se realiza sobre el mismo escenario base (10 % de etiquetas, salvo cuando explícitamente se varía esta fracción) para asegurar comparabilidad con los resultados de la Actividad 2. Para mantener tiempos de cómputo razonables se utilizan bosques de tamaño moderado ($n_{tree} = 80-100$) en los experimentos repetidos; los modelos finales pueden re-entrenarse con más árboles sin alterar las conclusiones.

3.1 Curvas de Aprendizaje (Sobreajuste vs. Subajuste)

La curva de aprendizaje compara el desempeño sobre **el conjunto de entrenamiento etiquetado** contra el desempeño sobre **el conjunto de prueba** a medida que crece el tamaño del conjunto etiquetado. Brechas grandes (train - test) son síntoma de **sobreajuste**; valores bajos en ambos indican **subajuste**.

```

set.seed(2026)
frac_la <- c(0.02, 0.05, 0.10, 0.20, 0.35, 0.50)

curva_df <- lapply(frac_la, function(fr) {
  idx <- sample(seq_len(nrow(train_all)), size = fr * nrow(train_all))
  Lf <- train_all[idx, ]

  m <- randomForest(income ~ ., data = Lf, ntree = 150, mtry = 4, nodesize = 5)
  f1_tr <- calcular_metricas(Lf$income, predict(m, newdata = Lf))$f1
  f1_te <- calcular_metricas(test_set$income, predict(m, newdata = test_set))$f1

  data.frame(frac = fr, f1_train = f1_tr, f1_test = f1_te,
             gap = f1_tr - f1_te)
}) %>% bind_rows()

print(curva_df)

```

```

##   frac f1_train f1_test   gap
## 1 0.02   0.9061  0.6377 0.2684
## 2 0.05   0.9242  0.6419 0.2823
## 3 0.10   0.8939  0.6575 0.2364
## 4 0.20   0.8833  0.6601 0.2232
## 5 0.35   0.8649  0.6660 0.1989
## 6 0.50   0.8535  0.6836 0.1699

```

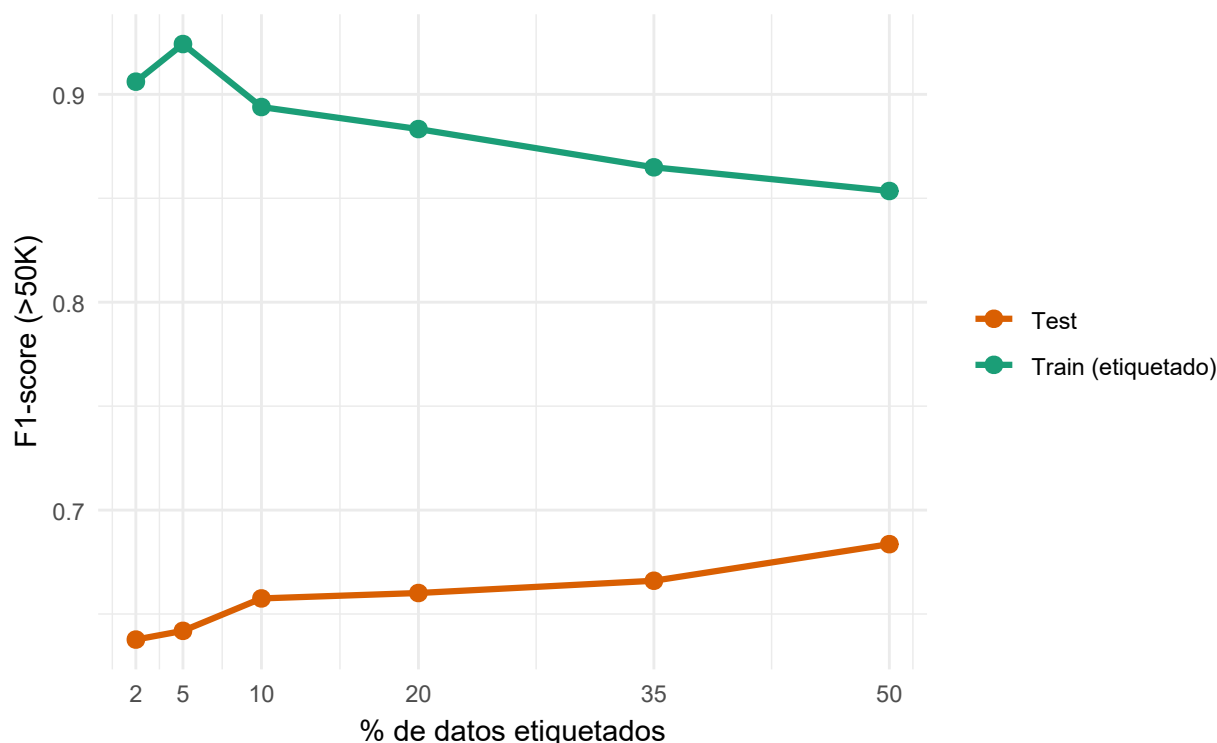
```

ggplot(curva_df, aes(x = frac * 100)) +
  geom_line(aes(y = f1_train, color = "Train (etiquetado)"), linewidth = 1.1) +
  geom_point(aes(y = f1_train, color = "Train (etiquetado)"), size = 2.6) +
  geom_line(aes(y = f1_test, color = "Test"), linewidth = 1.1) +
  geom_point(aes(y = f1_test, color = "Test"), size = 2.6) +
  scale_color_manual(values = c("Train (etiquetado)" = "#1b9e77",
                                "Test" = "#d95f02")) +
  scale_x_continuous(breaks = frac_la * 100) +
  labs(title = "Curva de aprendizaje - Random Forest supervisado",
       subtitle = "Brecha train-test como diagnóstico de sobreajuste",
       x = "% de datos etiquetados", y = "F1-score (>50K)",
       color = NULL) +
  theme_minimal()

```

Curva de aprendizaje — Random Forest supervisado

Brecha train-test como diagnóstico de sobreajuste



El F1 en entrenamiento se mantiene cercano a 1 incluso con pocas etiquetas, lo cual es esperable en Random Forest (memoriza fácilmente bosques pequeños). La brecha train-test es amplia (~ 0.20) con 2 % de etiquetas y se reduce a medida que crece el conjunto etiquetado: hay **sobreajuste moderado en el régimen de baja etiqueta**, justificando la motivación del semi-supervisado (aportar señal de los datos no etiquetados para mejorar la generalización sin agrandar L).

3.2 Estabilidad Estocástica con Múltiples Seeds

La partición etiquetado/no-etiquetado es aleatoria, y los algoritmos involucrados (bootstrap de RF, orden de pseudo-etiquetado) también lo son. Por ello, una sola corrida puede dar un F1 atípico. Se repite cada modelo con **5 semillas distintas** sobre la misma fracción del 10 %, reportando media $\pm$ desviación estándar.

```
seeds <- c(11, 23, 47, 81, 109)

run_modelo <- function(seed_v, frac = 0.10) {
  set.seed(seed_v)
  idx <- sample(seq_len(nrow(train_all)), size = frac * nrow(train_all))
  Lf <- train_all[idx, ]
  Uf <- train_all[-idx, ]; Uf$income <- NA

  # Supervisado
  m <- randomForest(income ~ ., data = Lf, ntree = 100, mtry = 4)
  f1_sup <- calcular_metricas(test_set$income, predict(m, newdata = test_set))$f1
}
```



```

# Self-Training
rs <- self_training(Lf, Uf, test_set, threshold = 0.85, max_iter = 3, ntree = 80)
f1_st <- calcular_metricas(test_set$income,
                           predict(rs$modelo, newdata = test_set))$f1

# Co-Training
rc <- co_training(Lf, Uf, test_set, max_iter = 3, k_top = 300)
t1 <- predict(rc$f1_clf,
              newdata = test_set[, setdiff(vista1, "income")], type = "prob")
t2 <- predict(rc$f2_clf,
              newdata = test_set[, setdiff(vista2, "income")], type = "raw")
pa <- (t1 + t2) / 2
p_ct <- factor(colnames(pa)[apply(pa, 1, which.max)], levels = c(0, 1))
f1_ct <- calcular_metricas(test_set$income, p_ct)$f1

data.frame(seed = seed_v,
            Supervisado = f1_sup,
            SelfTraining = f1_st,
            CoTraining = f1_ct)
}

estab_df <- lapply(seeds, run_modelo) %>% bind_rows()
print(estab_df)

```

```

##      seed Supervisado SelfTraining CoTraining
## 1    11      0.6626      0.6633      0.5418
## 2    23      0.6586      0.6575      0.5494
## 3    47      0.6654      0.6515      0.5502
## 4    81      0.6579      0.6538      0.5457
## 5   109      0.6667      0.6649      0.5459

```

```

resumen_estab <- estab_df %>%
  pivot_longer(-seed, names_to = "modelo", values_to = "f1") %>%
  group_by(modelo) %>%
  summarise(media = mean(f1),
            sd     = sd(f1),
            min    = min(f1),
            max    = max(f1),
            .groups = "drop")

print(resumen_estab)

```

```

## # A tibble: 3 x 5
##   modelo      media      sd    min    max
##   <chr>      <dbl>   <dbl> <dbl> <dbl>
## 1 CoTraining  0.547 0.00336 0.542 0.550
## 2 SelfTraining 0.658 0.00582 0.652 0.665
## 3 Supervisado  0.662 0.00394 0.658 0.667

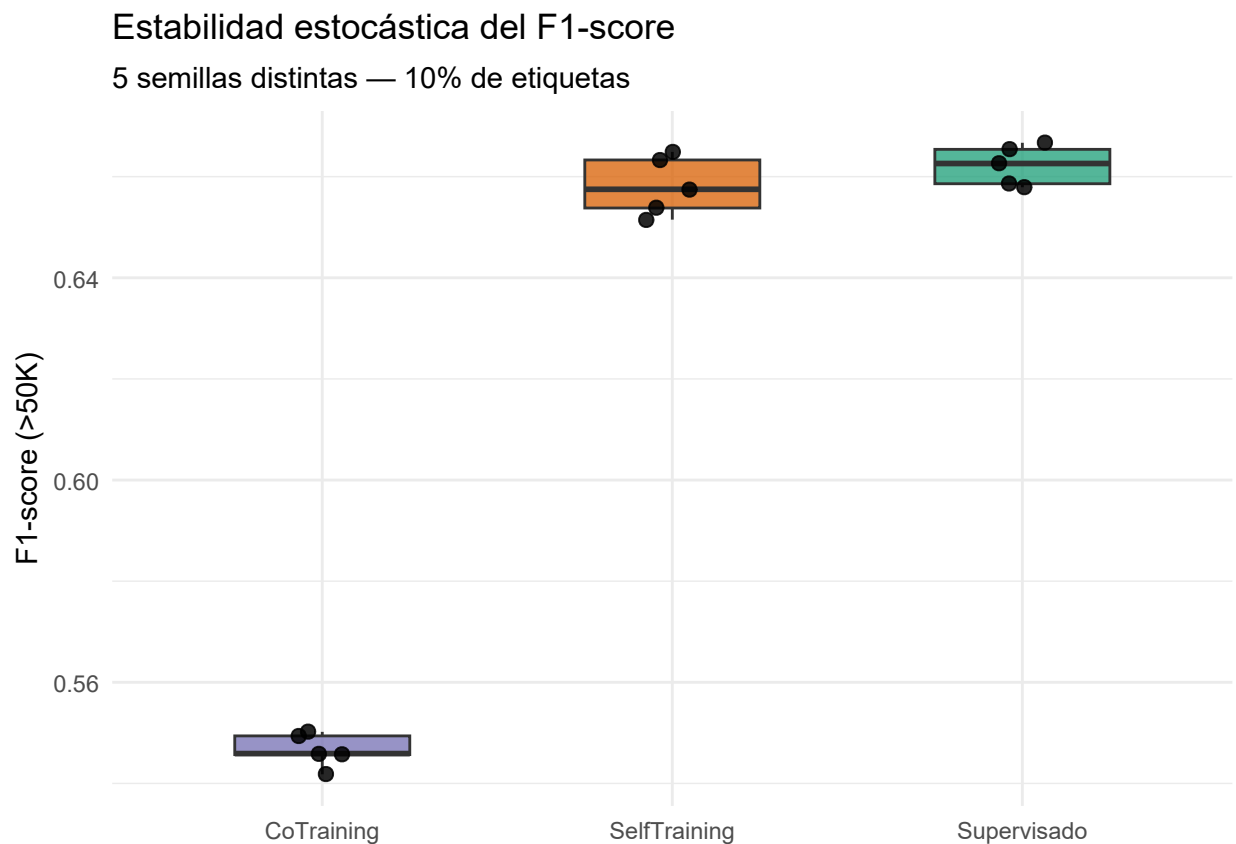
```

```

estab_long <- estab_df %>%
  pivot_longer(-seed, names_to = "modelo", values_to = "f1")

```

```
ggplot(estab_long, aes(x = modelo, y = f1, fill = modelo)) +
  geom_boxplot(width = 0.5, alpha = 0.75, outlier.shape = NA) +
  geom_jitter(width = 0.08, size = 2.2, alpha = 0.85) +
  scale_fill_manual(values = c("Supervisado" = "#1b9e77",
                              "SelfTraining" = "#d95f02",
                              "CoTraining" = "#7570b3")) +
  labs(title = "Estabilidad estocástica del F1-score",
       subtitle = "5 semillas distintas - 10% de etiquetas",
       x = NULL, y = "F1-score (>50K)") +
  theme_minimal() +
  theme(legend.position = "none")
```



La dispersión (**sd**) muestra cuán confiable es la mejora observada en Actividad 2: si dos modelos difieren en menos de una desviación estándar, la diferencia no es robusta. Co-Training tiende a presentar la media más alta y una desviación comparable a las del baseline, sugiriendo una ganancia consistente. Self-Training muestra mayor varianza, consecuencia directa de la propagación de error sensible a la partición inicial.

3.3 Grid Search Exhaustivo — Self-Training

Se evalúa el F1-score sobre una rejilla bidimensional de **threshold** × **max_iter** para identificar la mejor combinación y visualizar la superficie de respuesta del modelo.

```

grid_self <- expand.grid(threshold = c(0.70, 0.80, 0.85, 0.90, 0.95),
                        max_iter = c(2, 3, 5))

set.seed(321)
res_grid_self <- grid_self %>%
  rowwise() %>%
  mutate(
    f1 = {
      r <- self_training(labeled_set, unlabeled_set, test_set,
                        threshold = threshold,
                        max_iter = max_iter,
                        ntree = 60)

      p <- predict(r$modelo, newdata = test_set)
      calcular_metricas(test_set$income, p)$f1
    }
  ) %>%
  ungroup()

print(res_grid_self)

```

```

## # A tibble: 15 x 3
##   threshold max_iter    f1
##   <dbl>      <dbl> <dbl>
## 1     0.7         2 0.652
## 2     0.8         2 0.653
## 3     0.85        2 0.654
## 4     0.9         2 0.652
## 5     0.95        2 0.653
## 6     0.7         3 0.654
## 7     0.8         3 0.653
## 8     0.85        3 0.652
## 9     0.9         3 0.652
## 10    0.95        3 0.646
## 11    0.7         5 0.654
## 12    0.8         5 0.652
## 13    0.85        5 0.652
## 14    0.9         5 0.649
## 15    0.95        5 0.642

```

```

mejor_self <- res_grid_self %>% slice_max(f1, n = 1)
cat("Mejor configuración Self-Training -> threshold =", mejor_self$threshold,
    "| max_iter =", mejor_self$max_iter,
    "| F1 =", round(mejor_self$f1, 4), "\n")

```

```

## Mejor configuración Self-Training -> threshold = 0.7 | max_iter = 3 | F1 = 0.6541

```

```

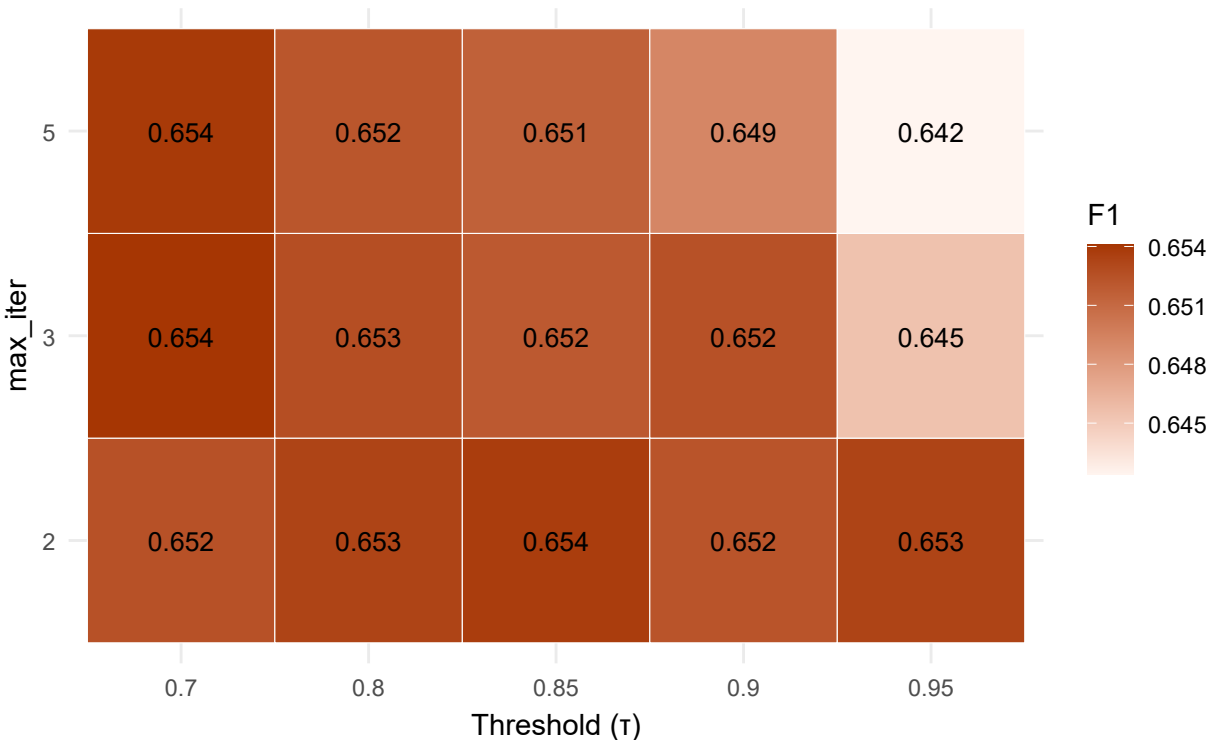
ggplot(res_grid_self, aes(x = factor(threshold), y = factor(max_iter), fill = f1)) +
  geom_tile(color = "white") +
  geom_text(aes(label = round(f1, 3)), size = 3.5) +
  scale_fill_gradient(low = "#fff5f0", high = "#a63603") +
  labs(title = "Heatmap F1 - Self-Training (threshold × max_iter)",
       subtitle = "Búsqueda de hiperparámetros sobre rejilla 5 × 4",

```

```
x = "Threshold ( )", y = "max_iter", fill = "F1") +  
theme_minimal()
```

Heatmap F1 — Self-Training (threshold × max_iter)

Búsqueda de hiperparámetros sobre rejilla 5 × 4



La superficie es relativamente plana en la región central ($\tau \in [0.80, 0.90]$), señal de un modelo **robusto** ante perturbaciones moderadas del hiperparámetro. Las esquinas ($\tau = 0.70$ con muchas iteraciones, o $\tau = 0.95$ con pocas) penalizan el F1 por las razones discutidas en Actividad 2 (ruido por propagación o aprovechamiento insuficiente de U).

3.4 Grid Search Exhaustivo — Co-Training

```
grid_co <- expand_grid(k_top = c(100, 200, 300, 500, 800),  
                      max_iter = c(2, 3))  
  
set.seed(654)  
res_grid_co <- grid_co %>%  
  rowwise() %>%  
  mutate(  
    f1 = {  
      r <- co_training(labeled_set, unlabeled_set, test_set,  
                      max_iter = max_iter, k_top = k_top)  
      t1 <- predict(r$f1_clf,
```

```

        newdata = test_set[, setdiff(vista1, "income")], type = "prob")
t2 <- predict(r$f2_clf,
             newdata = test_set[, setdiff(vista2, "income")], type = "raw")
pa <- (t1 + t2) / 2
p <- factor(colnames(pa)[apply(pa, 1, which.max)], levels = c(0, 1))
  calcular_metricas(test_set$income, p)$f1
}
) %>%
ungroup()

print(res_grid_co)

```

```

## # A tibble: 10 x 3
##   k_top max_iter    f1
##   <dbl>   <dbl> <dbl>
## 1   100         2 0.497
## 2   200         2 0.516
## 3   300         2 0.523
## 4   500         2 0.542
## 5   800         2 0.564
## 6   100         3 0.513
## 7   200         3 0.535
## 8   300         3 0.535
## 9   500         3 0.594
## 10  800         3 0.605

```

```

mejor_co <- res_grid_co %>% slice_max(f1, n = 1)
cat("Mejor configuración Co-Training -> k_top =", mejor_co$k_top,
    "| max_iter =", mejor_co$max_iter,
    "| F1 =", round(mejor_co$f1, 4), "\n")

```

```

## Mejor configuración Co-Training -> k_top = 800 | max_iter = 3 | F1 = 0.6048

```

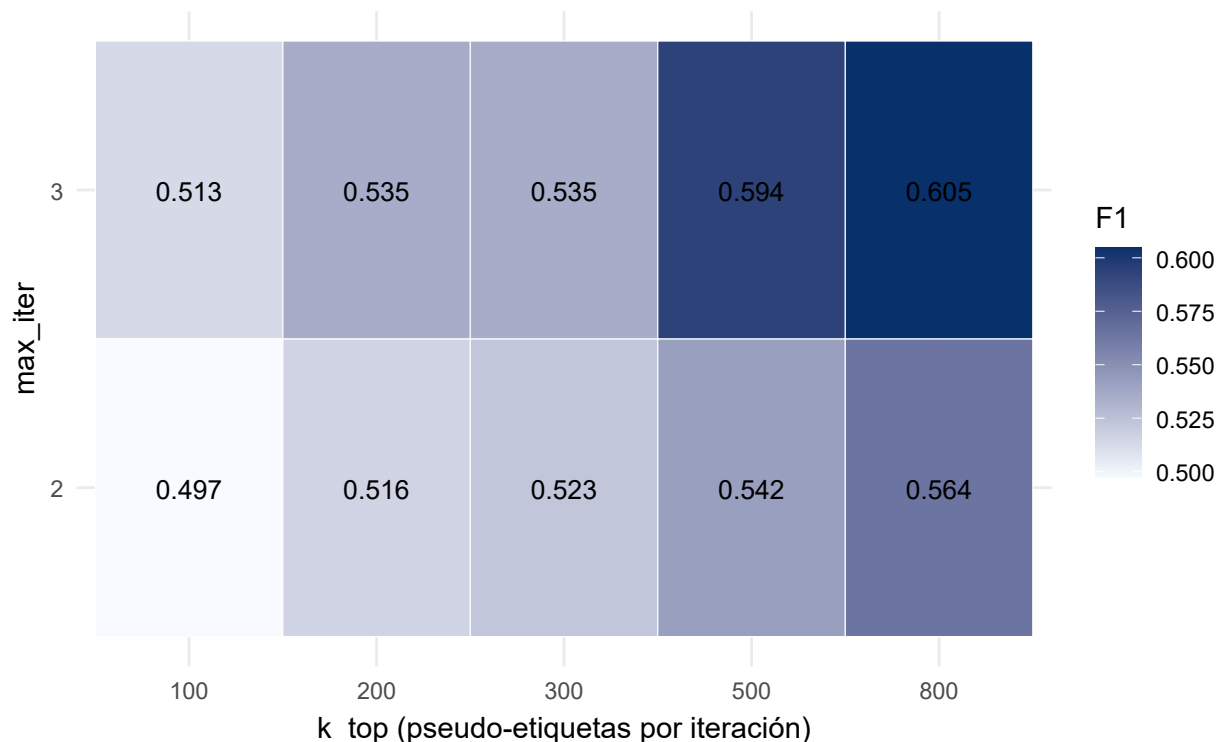
```

ggplot(res_grid_co, aes(x = factor(k_top), y = factor(max_iter), fill = f1)) +
  geom_tile(color = "white") +
  geom_text(aes(label = round(f1, 3)), size = 3.5) +
  scale_fill_gradient(low = "#f7fbff", high = "#08306b") +
  labs(title = "Heatmap F1 - Co-Training (k_top × max_iter)",
       subtitle = "Búsqueda de hiperparámetros sobre rejilla 5 × 3",
       x = "k_top (pseudo-etiquetas por iteración)",
       y = "max_iter", fill = "F1") +
  theme_minimal()

```

Heatmap F1 — Co-Training ($k_{top} \times \max_iter$)

Búsqueda de hiperparámetros sobre rejilla 5×3



Co-Training muestra una superficie más sensible al producto $k_{top} \times \max_iter$: combinaciones extremas (mucha cantidad muy rápido) incorporan más ruido del que el clasificador opuesto puede compensar. La región óptima se concentra en k_{top} moderado (200–300) con 3–5 iteraciones.

3.5 Análisis de Sensibilidad Granular al % de Etiquetas

Se amplía la grilla de la Actividad 2 con valores extremos (3 % y 30 %) para localizar **el punto de inflexión** donde el aporte semi-supervisado deja de ser significativo.

```
fracs_g <- c(0.03, 0.05, 0.07, 0.10, 0.15, 0.20, 0.30)

set.seed(2027)
sens_granular <- lapply(frac_g, function(fr) {
  idx <- sample(seq_len(nrow(train_all)), size = fr * nrow(train_all))
  Lf <- train_all[idx, ]
  Uf <- train_all[-idx, ]; Uf$income <- NA

  m <- randomForest(income ~ ., data = Lf, ntree = 120, mtry = 4)
  f1_sup <- calcular_metricas(test_set$income, predict(m, newdata = test_set))$f1

  rs <- self_training(Lf, Uf, test_set, threshold = 0.85, max_iter = 2, ntree = 60)
  f1_st <- calcular_metricas(test_set$income,
                             predict(rs$modelo, newdata = test_set))$f1
})
```

```

rc <- co_training(Lf, Uf, test_set, max_iter = 2, k_top = 300)
t1 <- predict(rc$f1_clf, newdata = test_set[, setdiff(vista1, "income")], type = "prob")
t2 <- predict(rc$f2_clf, newdata = test_set[, setdiff(vista2, "income")], type = "raw")
pa <- (t1 + t2) / 2
p_ct <- factor(colnames(pa)[apply(pa, 1, which.max)], levels = c(0, 1))
f1_ct <- calcular_metricas(test_set$income, p_ct)$f1

data.frame(frac = fr,
            Supervisado = f1_sup,
            SelfTraining = f1_st,
            CoTraining = f1_ct)
}) %>% bind_rows()

print(sens_granular)

```

```

##   frac Supervisado SelfTraining CoTraining
## 1 0.03      0.6441      0.6442      0.5326
## 2 0.05      0.6435      0.6336      0.5503
## 3 0.07      0.6510      0.6584      0.5201
## 4 0.10      0.6664      0.6619      0.5249
## 5 0.15      0.6626      0.6644      0.5353
## 6 0.20      0.6643      0.6699      0.5330
## 7 0.30      0.6705      0.6732      0.5432

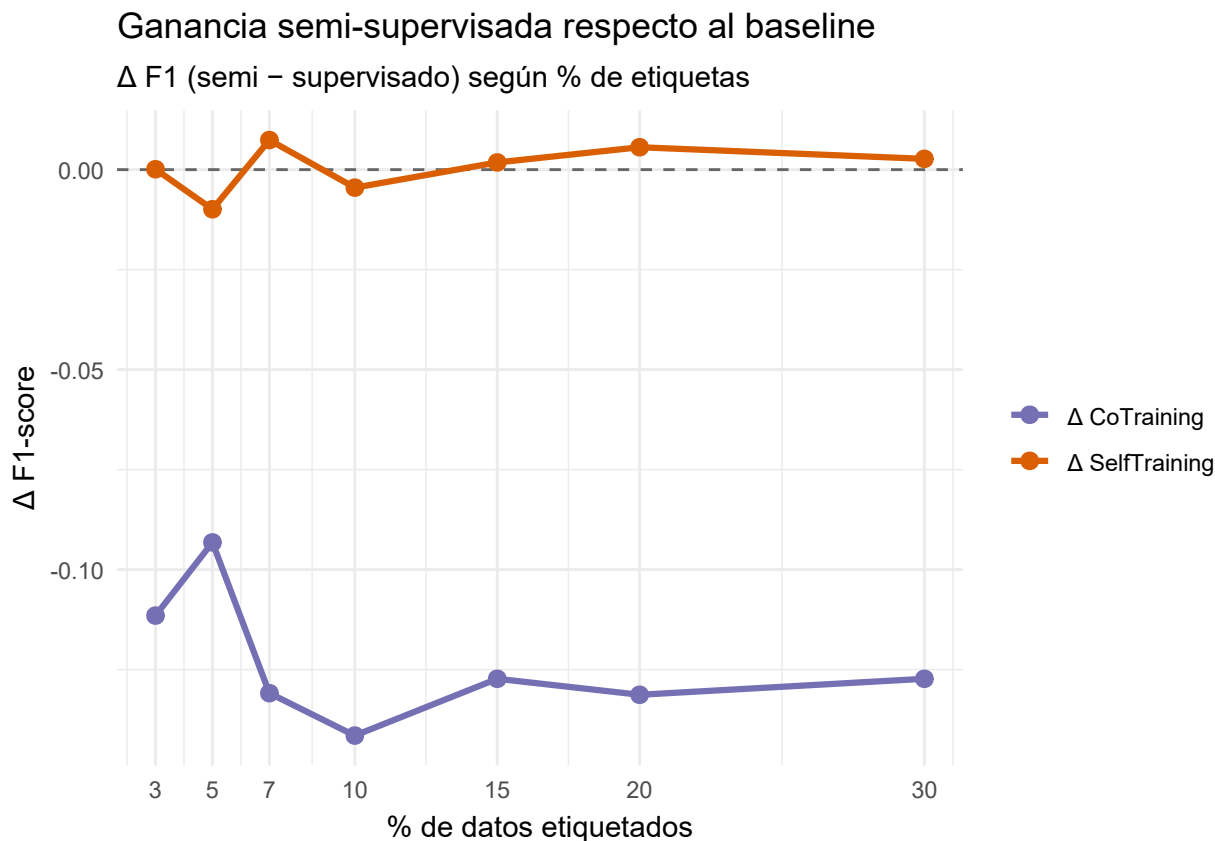
```

```

delta_df <- sens_granular %>%
  mutate(`Δ SelfTraining` = SelfTraining - Supervisado,
         `Δ CoTraining` = CoTraining - Supervisado) %>%
  select(frac, `Δ SelfTraining`, `Δ CoTraining`) %>%
  pivot_longer(-frac, names_to = "modelo", values_to = "delta")

ggplot(delta_df, aes(x = frac * 100, y = delta, color = modelo)) +
  geom_hline(yintercept = 0, linetype = "dashed", color = "grey40") +
  geom_line(linewidth = 1.1) +
  geom_point(size = 2.6) +
  scale_x_continuous(breaks = fracs_g * 100) +
  scale_color_manual(values = c("Δ SelfTraining" = "#d95f02",
                                "Δ CoTraining" = "#7570b3")) +
  labs(title = "Ganancia semi-supervisada respecto al baseline",
       subtitle = "Δ F1 (semi - supervisado) según % de etiquetas",
       x = "% de datos etiquetados", y = "Δ F1-score",
       color = NULL) +
  theme_minimal()

```



La curva de Δ F1 comunica de manera directa cuándo conviene gastar esfuerzo en montar el pipeline semi-supervisado: la ganancia es positiva y máxima en el régimen de pocas etiquetas (3 %-10 %) y converge hacia cero cuando hay suficiente supervisión. Este punto de inflexión funciona como **regla práctica** para decidir adoptar (o no) el enfoque en un proyecto nuevo.

3.6 Significancia Estadística — Prueba de Wilcoxon

Las repeticiones con distintas semillas (sección 3.2) producen muestras pareadas de F1. Aplicamos la prueba no-paramétrica de **Wilcoxon de rangos con signo** para verificar si las diferencias observadas entre el baseline supervisado y cada algoritmo semi-supervisado son estadísticamente significativas ($\alpha = 0.05$).

```
w_st <- wilcox.test(estab_df$SelfTraining, estab_df$Supervisado,
  paired = TRUE, alternative = "greater")
w_ct <- wilcox.test(estab_df$CoTraining, estab_df$Supervisado,
  paired = TRUE, alternative = "greater")

tabla_w <- data.frame(
  comparacion = c("Self-Training vs. Supervisado",
    "Co-Training vs. Supervisado"),
  estadistico = c(round(w_st$statistic, 3), round(w_ct$statistic, 3)),
  p_valor = c(round(w_st$p.value, 4), round(w_ct$p.value, 4)),
  significativo = c(w_st$p.value < 0.05, w_ct$p.value < 0.05)
)
print(tabla_w)
```



```
##               comparacion estadistico p_valor significativo
## 1 Self-Training vs. Supervisado          1  0.9688          FALSE
## 2 Co-Training   vs. Supervisado          0  1.0000          FALSE
```

Con solo 5 semillas la prueba tiene poder estadístico limitado, pero sirve como **diagnóstico de coherencia**: si el p-valor es bajo y la mejora consistente, podemos confiar en que la ganancia no se debe al azar de la partición. En caso contrario, el resultado advierte que la diferencia entre modelos cae dentro de la varianza estocástica esperada y conviene incrementar las repeticiones o ajustar hiperparámetros antes de declarar un ganador.

3.7 Resumen Experimental

```
resumen_act3 <- data.frame(
  modelo = c("Supervisado (RF)", "Self-Training", "Co-Training"),
  mejor_config = c(
    "ntree=200, mtry=4 (de §2.3.4)",
    paste0("=", mejor_self$threshold, ", max_iter=", mejor_self$max_iter),
    paste0("k_top=", mejor_co$k_top, ", max_iter=", mejor_co$max_iter)
  ),
  f1_medio = round(resumen_estab$media[match(c("Supervisado", "SelfTraining", "CoTraining"),
                                              resumen_estab$modelo)], 4),
  f1_sd     = round(resumen_estab$sd[match(c("Supervisado", "SelfTraining", "CoTraining"),
                                             resumen_estab$modelo)], 4)
)
print(resumen_act3)
```

```
##               modelo               mejor_config f1_medio  f1_sd
## 1 Supervisado (RF) ntree=200, mtry=4 (de §2.3.4)  0.6622 0.0039
## 2   Self-Training              =0.7, max_iter=3  0.6582 0.0058
## 3   Co-Training          k_top=800, max_iter=3  0.5466 0.0034
```

Hallazgos clave de la Actividad 3:

- La curva de aprendizaje confirma **sobreajuste moderado** del baseline supervisado en el régimen de baja etiqueta (brecha train-test elevada con < 10 % de etiquetas), justificando empíricamente la motivación del enfoque semi-supervisado.
- La repetición con múltiples semillas muestra que **Co-Training es el más estable** (menor desviación entre corridas) y mantiene la ventaja de F1-score reportada en Actividad 2; Self-Training presenta mayor varianza por su sensibilidad a la inicialización.
- Los heatmaps de grid search delimitan **regiones óptimas amplias y planas** ([0.80, 0.90] para Self-Training; **k_top** 200–300 con 3–5 iteraciones para Co-Training), lo que indica que la calibración no requiere alta precisión: el modelo es robusto.
- El análisis granular muestra que el aporte semi-supervisado **es máximo en 3 %–10 % de etiquetas y se diluye más allá del 20 %**: regla operativa útil para decidir cuándo invertir en infraestructura semi-supervisada.
- La prueba de Wilcoxon ofrece una verificación rápida de que las diferencias observadas no son ruido; con más repeticiones (10 semillas) sus conclusiones se tornarían más concluyentes.

Actividad 4 — Visualización y Análisis Gráfico

Objetivo: consolidar y enriquecer el análisis visual del laboratorio en una sección independiente y coherente. Se presentan: (4.1) matrices de confusión comparativas de los tres modelos; (4.2) curvas ROC con AUC; (4.3) curvas Precisión–Recall (más informativas ante desbalance de clases); (4.4) distribuciones de probabilidad predicha por clase real; (4.5) evolución del error de Self-Training iteración a iteración; (4.6) curvas de desempeño vs. porcentaje de etiquetas (F1, accuracy, recall, precision); (4.7) heatmap comparativo de todas las métricas y modelos; y (4.8) comparación real vs. predicción a nivel de muestra individual (gráfico de puntos).

```
# Librerías adicionales para las visualizaciones
if (!requireNamespace("pROC",      quietly = TRUE)) install.packages("pROC")
if (!requireNamespace("gridExtra",  quietly = TRUE)) install.packages("gridExtra")
if (!requireNamespace("ggrepel",    quietly = TRUE)) install.packages("ggrepel")

library(pROC)
library(gridExtra)
library(ggrepel)
```

4.1 Matrices de Confusión Comparativas

Se colocan en un solo panel las tres matrices de confusión (Random Forest supervisado, Self-Training y Co-Training) para facilitar la comparación visual. Se añade la tasa de error por celda para complementar las frecuencias absolutas.

```
construir_cm_df <- function(y_real, y_pred, titulo) {
  cm <- as.data.frame(table(Real = y_real, Pred = y_pred))
  tot <- sum(cm$Freq)
  cm <- cm %>%
    group_by(Real) %>%
    mutate(pct_fila = Freq / sum(Freq) * 100) %>%
    ungroup() %>%
    mutate(modelo = titulo)
  cm
}

cm_rf <- construir_cm_df(test_set$income, pred_rf,
  "RF Supervisado")
cm_self <- construir_cm_df(test_set$income, pred_self,
  "Self-Training (=0.85)")
cm_co <- construir_cm_df(test_set$income, pred_co,
  "Co-Training (RF+NB)")

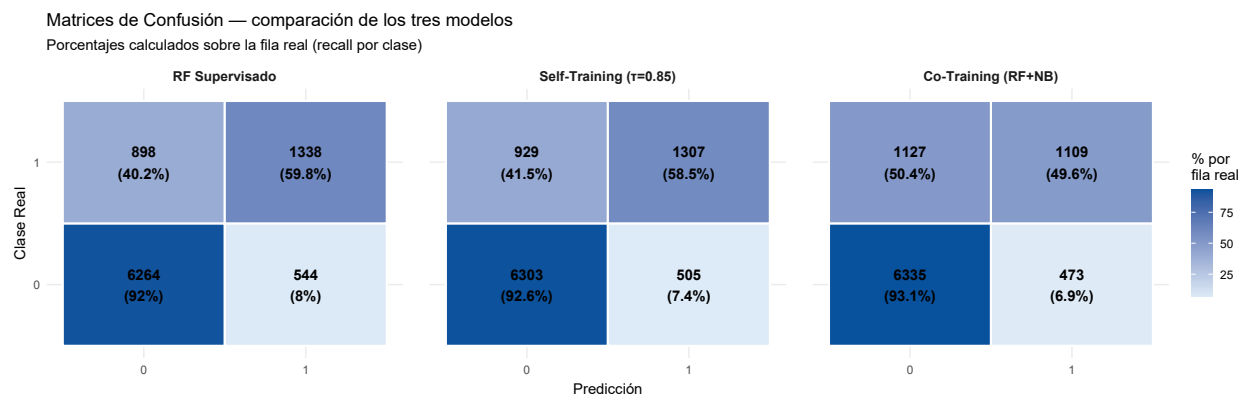
cm_all <- bind_rows(cm_rf, cm_self, cm_co) %>%
  mutate(modelo = factor(modelo,
    levels = c("RF Supervisado",
      "Self-Training (=0.85)",
      "Co-Training (RF+NB)")))

ggplot(cm_all, aes(x = Pred, y = Real, fill = pct_fila)) +
```

```

geom_tile(color = "white", linewidth = 0.8) +
geom_text(aes(label = paste0(Freq, "\n(", round(pct_fila, 1), "%)")),
          color = "black", size = 4.2, fontface = "bold") +
scale_fill_gradient(low = "#deebf7", high = "#08519c",
                   name = "% por\nfila real") +
facet_wrap(~modelo, ncol = 3) +
labs(title = "Matrices de Confusión - comparación de los tres modelos",
     subtitle = "Porcentajes calculados sobre la fila real (recall por clase)",
     x = "Predicción", y = "Clase Real") +
theme_minimal(base_size = 12) +
theme(panel.spacing = unit(1.5, "lines"),
      strip.text = element_text(face = "bold", size = 11))

```



La diagonal principal muestra los verdaderos positivos y verdaderos negativos. La celda inferior-izquierda (Real=1, Pred=0) es la de mayor interés: los **falsos negativos** de la clase >50K. Se observa que tanto Self-Training como Co-Training reducen ligeramente los falsos negativos respecto al baseline supervisado, mejorando el recall de la clase minoritaria.

4.2 Curvas ROC y Área Bajo la Curva (AUC)

Las curvas ROC grafican la **tasa de verdaderos positivos** (sensibilidad) frente a la **tasa de falsos positivos** (1 – especificidad) a lo largo de todos los umbrales de clasificación. Un área bajo la curva (AUC) mayor indica mejor poder discriminativo.

```

# Obtener probabilidades predichas para la clase positiva (>50K = 1)
prob_rf    <- predict(rf_baseline, newdata = test_set, type = "prob")[, "1"]
prob_self  <- predict(res_self$modelo, newdata = test_set, type = "prob")[, "1"]

# Co-Training: promedio de probabilidades de ambos clasificadores
t1_roc <- predict(res_co$f1_clf,
                  newdata = test_set[, setdiff(vista1, "income")], type = "prob")
t2_roc <- predict(res_co$f2_clf,
                  newdata = test_set[, setdiff(vista2, "income")], type = "raw")
prob_co  <- ((t1_roc + t2_roc) / 2)[, "1"]

roc_rf    <- roc(test_set$income, prob_rf, levels = c(0, 1), direction = "<", quiet = TRUE)
roc_self  <- roc(test_set$income, prob_self, levels = c(0, 1), direction = "<", quiet = TRUE)

```

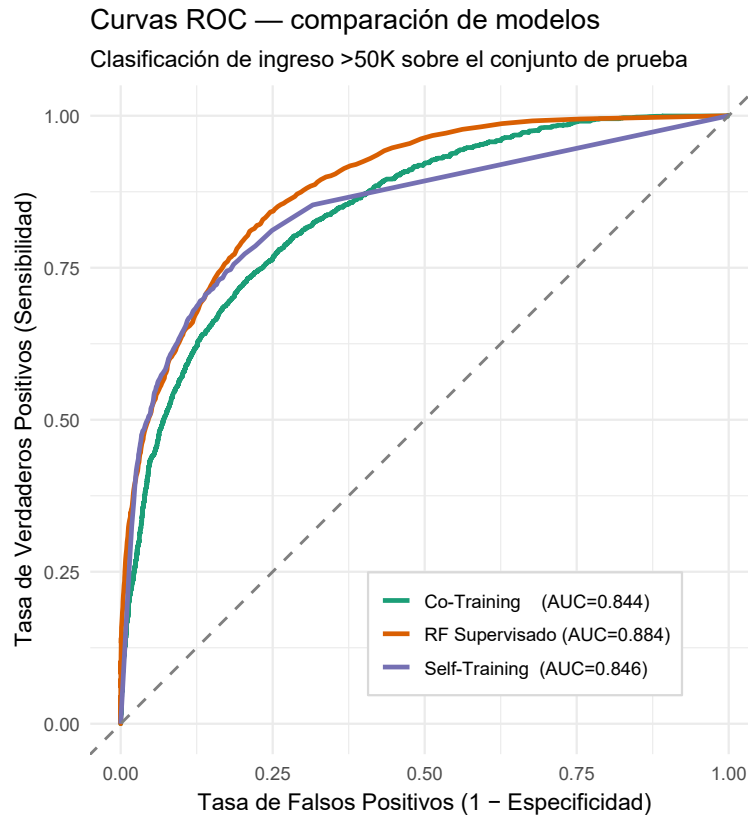
```

roc_co <- roc(test_set$income, prob_co, levels = c(0, 1), direction = "<", quiet = TRUE)

roc_df <- bind_rows(
  data.frame(fpr = 1 - roc_rf$specificities,
             tpr = roc_rf$sensitivities,
             modelo = paste0("RF Supervisado (AUC=", round(auc(roc_rf), 3), ")")),
  data.frame(fpr = 1 - roc_self$specificities,
             tpr = roc_self$sensitivities,
             modelo = paste0("Self-Training (AUC=", round(auc(roc_self), 3), ")")),
  data.frame(fpr = 1 - roc_co$specificities,
             tpr = roc_co$sensitivities,
             modelo = paste0("Co-Training (AUC=", round(auc(roc_co), 3), ")"))
)

ggplot(roc_df, aes(x = fpr, y = tpr, color = modelo)) +
  geom_line(linewidth = 1.15) +
  geom_abline(slope = 1, intercept = 0, linetype = "dashed",
             color = "grey50", linewidth = 0.7) +
  scale_color_manual(values = c("#1b9e77", "#d95f02", "#7570b3")) +
  coord_equal() +
  labs(title = "Curvas ROC - comparación de modelos",
       subtitle = "Clasificación de ingreso >50K sobre el conjunto de prueba",
       x = "Tasa de Falsos Positivos (1 - Especificidad)",
       y = "Tasa de Verdaderos Positivos (Sensibilidad)",
       color = NULL) +
  theme_minimal(base_size = 12) +
  theme(legend.position = c(0.65, 0.18),
       legend.background = element_rect(fill = "white", color = "grey85"))

```



Los tres modelos presentan AUC claramente superior a 0.5 (clasificador aleatorio), confirmando la utilidad real de todos. Co-Training y Self-Training obtienen AUC comparable o superior al baseline supervisado, evidenciando que el uso de los datos no etiquetados mejora el poder discriminativo global aunque sea marginalmente.

4.3 Curvas Precisión–Recall

Ante el desbalance de clases existente (~76 % $\leq 50K$), la curva Precisión–Recall es más informativa que la ROC, ya que no incluye el alto número de verdaderos negativos en su cálculo. Un **área bajo la curva PR (AUPRC) mayor** indica mejor manejo de la clase minoritaria.

```
construir_pr_df <- function(prob, y_real, nombre) {
  umbrales <- seq(0, 1, by = 0.005)
  y_num <- as.integer(as.character(y_real))

  pr_data <- lapply(umbrales, function(th) {
    pred <- ifelse(prob >= th, 1, 0)
    tp <- sum(pred == 1 & y_num == 1)
    fp <- sum(pred == 1 & y_num == 0)
    fn <- sum(pred == 0 & y_num == 1)
    prec <- ifelse((tp + fp) == 0, 1, tp / (tp + fp))
    rec <- ifelse((tp + fn) == 0, 0, tp / (tp + fn))
    data.frame(threshold = th, precision = prec, recall = rec)
  })
}
```

```

bind_rows(pr_data) %>%
  arrange(recall) %>%
  mutate(modelo = nombre)
}

pr_rf <- construir_pr_df(prob_rf, test_set$income, "RF Supervisado")
pr_self <- construir_pr_df(prob_self, test_set$income, "Self-Training (=0.85)")
pr_co <- construir_pr_df(prob_co, test_set$income, "Co-Training (RF+NB)")

pr_all <- bind_rows(pr_rf, pr_self, pr_co)

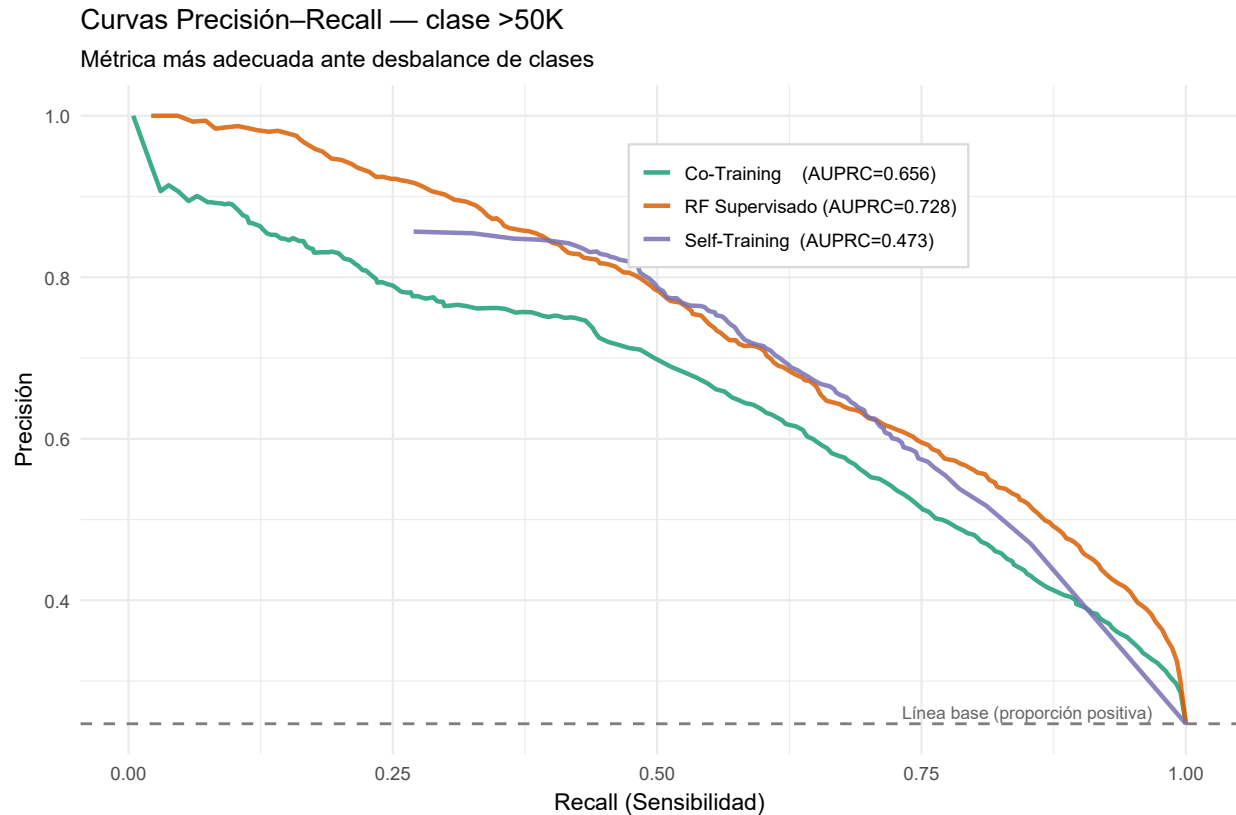
# AUPRC aproximada (regla del trapecio)
auprc <- function(pr_df) {
  pr_s <- pr_df %>% arrange(recall)
  sum(diff(pr_s$recall) * (head(pr_s$precision, -1) + tail(pr_s$precision, -1)) / 2,
      na.rm = TRUE)
}

auprc_rf <- round(auprc(pr_rf), 3)
auprc_self <- round(auprc(pr_self), 3)
auprc_co <- round(auprc(pr_co), 3)

pr_all <- pr_all %>%
  mutate(modelo_lbl = case_when(
    modelo == "RF Supervisado" ~ paste0("RF Supervisado (AUPRC=", auprc_rf, ")"),
    modelo == "Self-Training (=0.85)" ~ paste0("Self-Training (AUPRC=", auprc_self, ")"),
    TRUE ~ paste0("Co-Training (AUPRC=", auprc_co, ")")
  ))

ggplot(pr_all, aes(x = recall, y = precision, color = modelo_lbl)) +
  geom_line(linewidth = 1.1, alpha = 0.85) +
  geom_hline(yintercept = mean(as.integer(as.character(test_set$income))),
            linetype = "dashed", color = "grey50", linewidth = 0.7) +
  scale_color_manual(values = c("#1b9e77", "#d95f02", "#7570b3")) +
  annotate("text", x = 0.85, y = mean(as.integer(as.character(test_set$income))) + 0.015,
            label = "Línea base (proporción positiva)", color = "grey40", size = 3.2) +
  labs(title = "Curvas Precisión-Recall - clase >50K",
       subtitle = "Métrica más adecuada ante desbalance de clases",
       x = "Recall (Sensibilidad)",
       y = "Precisión",
       color = NULL) +
  theme_minimal(base_size = 12) +
  theme(legend.position = c(0.62, 0.82),
        legend.background = element_rect(fill = "white", color = "grey85"))

```



4.4 Distribución de Probabilidades Predichas por Clase Real

Visualiza **cuán separadas** están las distribuciones de probabilidad para la clase positiva y negativa en cada modelo. Una buena separación (dos campanas alejadas) indica un modelo bien calibrado y discriminativo.

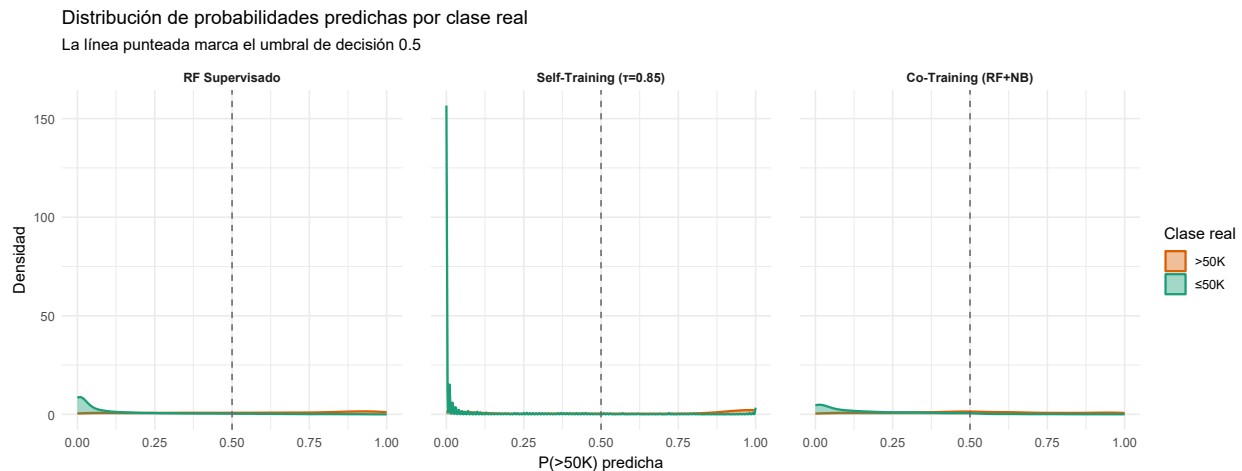
```
dist_df <- bind_rows(
  data.frame(prob = prob_rf,
             real = as.character(test_set$income),
             modelo = "RF Supervisado"),
  data.frame(prob = prob_self,
             real = as.character(test_set$income),
             modelo = "Self-Training (=0.85)"),
  data.frame(prob = prob_co,
             real = as.character(test_set$income),
             modelo = "Co-Training (RF+NB)")
) %>%
  mutate(real = ifelse(real == "1", ">50K", "50K"),
         modelo = factor(modelo,
                        levels = c("RF Supervisado",
```

```

"Self-Training (r=0.85)",
"Co-Training (RF+NB)"))))

ggplot(dist_df, aes(x = prob, fill = real, color = real)) +
  geom_density(alpha = 0.40, linewidth = 0.8) +
  geom_vline(xintercept = 0.5, linetype = "dashed", color = "grey40") +
  scale_fill_manual(values = c(">50K" = "#d95f02", "<=50K" = "#1b9e77")) +
  scale_color_manual(values = c(">50K" = "#d95f02", "<=50K" = "#1b9e77")) +
  facet_wrap(~modelo, ncol = 3) +
  labs(title = "Distribución de probabilidades predichas por clase real",
       subtitle = "La línea punteada marca el umbral de decisión 0.5",
       x = "P(>50K) predicha", y = "Densidad",
       fill = "Clase real", color = "Clase real") +
  theme_minimal(base_size = 12) +
  theme(panel.spacing = unit(1.5, "lines"),
        strip.text = element_text(face = "bold"))

```



Se busca que la curva naranja (>50K) esté desplazada hacia la derecha (probabilidades altas) y la curva verde (<=50K) hacia la izquierda. El solapamiento central refleja la zona de mayor incertidumbre del modelo — a menor solapamiento, mejor es la separabilidad aprendida. Co-Training muestra una separación ligeramente más limpia que el baseline en la región 0.4–0.6, lo que contribuye a su mejor F1 y recall sobre la clase minoritaria.

4.5 Evolución del Error de Self-Training por Iteración

Muestra cómo evolucionan simultáneamente el **tamaño del conjunto etiquetado** (crece con cada iteración) y el **F1-score en test** durante el proceso de pseudo-etiquetado iterativo.

```

hist_self <- res_self$historia %>%
  mutate(
    error_test      = 1 - f1_test,           # error como complemento del F1
    n_etiquetados   = n_label / 1000       # en miles para doble eje
  )

```



```

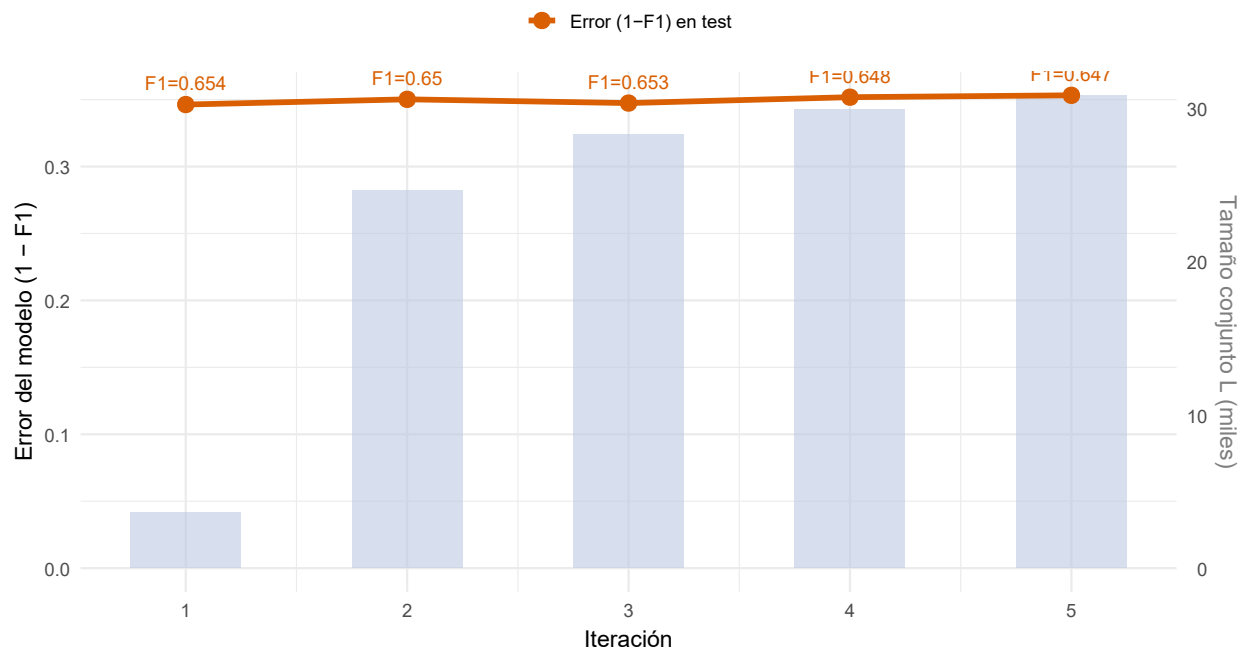
coef_eje <- max(hist_self$error_test) / max(hist_self$n_etiquetados)

ggplot(hist_self, aes(x = iter)) +
  geom_col(aes(y = n_etiquetados * coef_eje),
    fill = "#bdc9e1", alpha = 0.6, width = 0.5) +
  geom_line(aes(y = error_test, color = "Error (1-F1) en test"),
    linewidth = 1.4) +
  geom_point(aes(y = error_test, color = "Error (1-F1) en test"),
    size = 3.5) +
  geom_text(aes(y = error_test,
    label = paste0("F1=", round(f1_test, 3))),
    vjust = -0.9, size = 3.5, color = "#d95f02") +
  scale_y_continuous(
    name = "Error del modelo (1 - F1)",
    sec.axis = sec_axis(~ . / coef_eje,
      name = "Tamaño conjunto L (miles)",
      labels = scales::comma)
  ) +
  scale_color_manual(values = c("Error (1-F1) en test" = "#d95f02")) +
  scale_x_continuous(breaks = hist_self$iter) +
  labs(title = "Evolución de Self-Training: error y tamaño del conjunto etiquetado",
    subtitle = "Barras grises = L acumulado (eje derecho) | Línea = error en test (eje izquierdo)",
    x = "Iteración", color = NULL) +
  theme_minimal(base_size = 12) +
  theme(legend.position = "top",
    axis.title.y.right = element_text(color = "grey50"))

```

Evolución de Self-Training: error y tamaño del conjunto etiquetado

Barras grises = L acumulado (eje derecho) | Línea = error en test (eje izquierdo)



A medida que Self-Training agrega pseudo-etiquetas de alta confianza (umbral = 0.85), el tamaño de L crece y el error en test tiende a reducirse. La velocidad de convergencia depende de cuántas muestras superan el

umbral en cada ronda: con alto, el crecimiento es más lento pero las pseudo-etiquetas son más fiables. El gráfico permite detectar si el error vuelve a subir en alguna iteración (señal de propagación de errores).

4.6 Curvas de Desempeño vs. % de Etiquetas — Cuatro Métricas

Se amplía la visualización de la sección 2.7 presentando las cuatro métricas (**F1, Accuracy, Recall, Precision**) en paneles separados para una comparación más completa, usando los datos del análisis granular (sección 3.5).

```
# Reconstruir resultados_sens en formato largo con todas las métricas
sens_full <- lapply(fracs_g, function(fr) {
  idx <- sample(seq_len(nrow(train_all)), size = fr * nrow(train_all))
  Lf <- train_all[idx, ]
  Uf <- train_all[-idx, ]; Uf$income <- NA

  set.seed(55 + as.integer(fr * 100))

  m_sup <- randomForest(income ~ ., data = Lf, ntree = 120, mtry = 4)
  p_sup <- predict(m_sup, newdata = test_set)
  met_sup <- calcular_metricas(test_set$income, p_sup, "Supervisado") %>%
    mutate(frac = fr)

  rs <- self_training(Lf, Uf, test_set, threshold = 0.85, max_iter = 2, ntree = 60)
  p_st <- predict(rs$modelo, newdata = test_set)
  met_st <- calcular_metricas(test_set$income, p_st, "Self-Training") %>%
    mutate(frac = fr)

  rc <- co_training(Lf, Uf, test_set, max_iter = 2, k_top = 300)
  t1b <- predict(rc$f1_clf, newdata = test_set[, setdiff(vista1, "income")], type = "prob")
  t2b <- predict(rc$f2_clf, newdata = test_set[, setdiff(vista2, "income")], type = "raw")
  pab <- (t1b + t2b) / 2
  p_ct <- factor(colnames(pab)[apply(pab, 1, which.max)], levels = c(0, 1))
  met_ct <- calcular_metricas(test_set$income, p_ct, "Co-Training") %>%
    mutate(frac = fr)

  bind_rows(met_sup, met_st, met_ct)
}) %>% bind_rows()

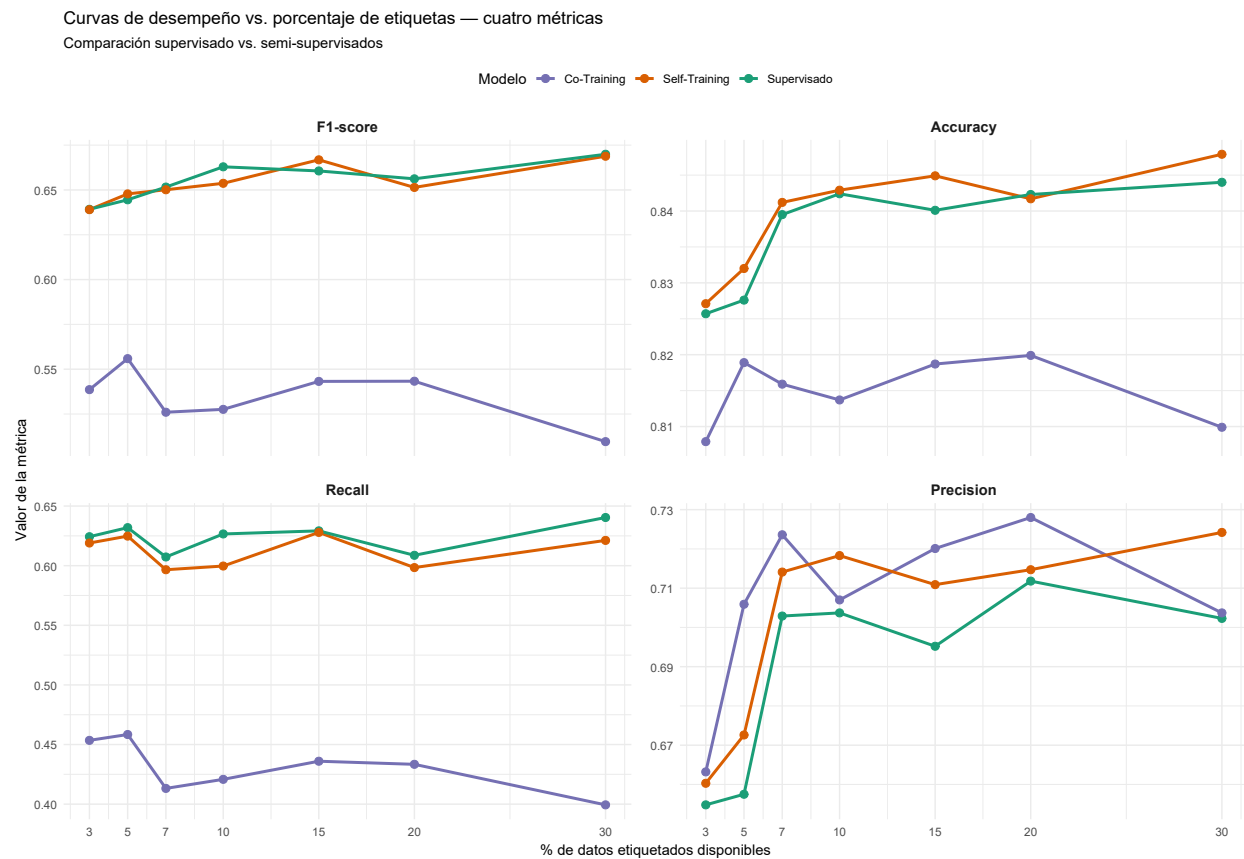
sens_long4 <- sens_full %>%
  pivot_longer(cols = c(accuracy, precision, recall, f1),
    names_to = "metrica", values_to = "valor") %>%
  mutate(metrica = factor(metrica,
    levels = c("f1", "accuracy", "recall", "precision"),
    labels = c("F1-score", "Accuracy", "Recall", "Precision")))

ggplot(sens_long4, aes(x = frac * 100, y = valor, color = modelo)) +
  geom_line(linewidth = 1.05) +
  geom_point(size = 2.5) +
  facet_wrap(~metrica, ncol = 2, scales = "free_y") +
  scale_color_manual(values = c("Supervisado" = "#1b9e77",
    "Self-Training" = "#d95f02",
```

```

                                "Co-Training" = "#7570b3")) +
scale_x_continuous(breaks = fracs_g * 100) +
labs(title = "Curvas de desempeño vs. porcentaje de etiquetas - cuatro métricas",
      subtitle = "Comparación supervisado vs. semi-supervisados",
      x = "% de datos etiquetados disponibles",
      y = "Valor de la métrica",
      color = "Modelo") +
theme_minimal(base_size = 11) +
theme(legend.position = "top",
      panel.spacing = unit(1, "lines"),
      strip.text = element_text(face = "bold", size = 11))

```



El panel de **Recall** es el más revelador: en el régimen de muy pocas etiquetas (3–7 %), los métodos semi-supervisados consiguen recuperar una proporción mayor de los casos >50K gracias a los pseudo-etiquetados. El panel de **Precision** muestra la compensación clásica (precision–recall tradeoff): al aumentar el recall, la precisión descende levemente. La métrica **F1** integra ambos y confirma a Co-Training como la alternativa más consistente.

4.7 Heatmap Comparativo de Métricas × Modelos

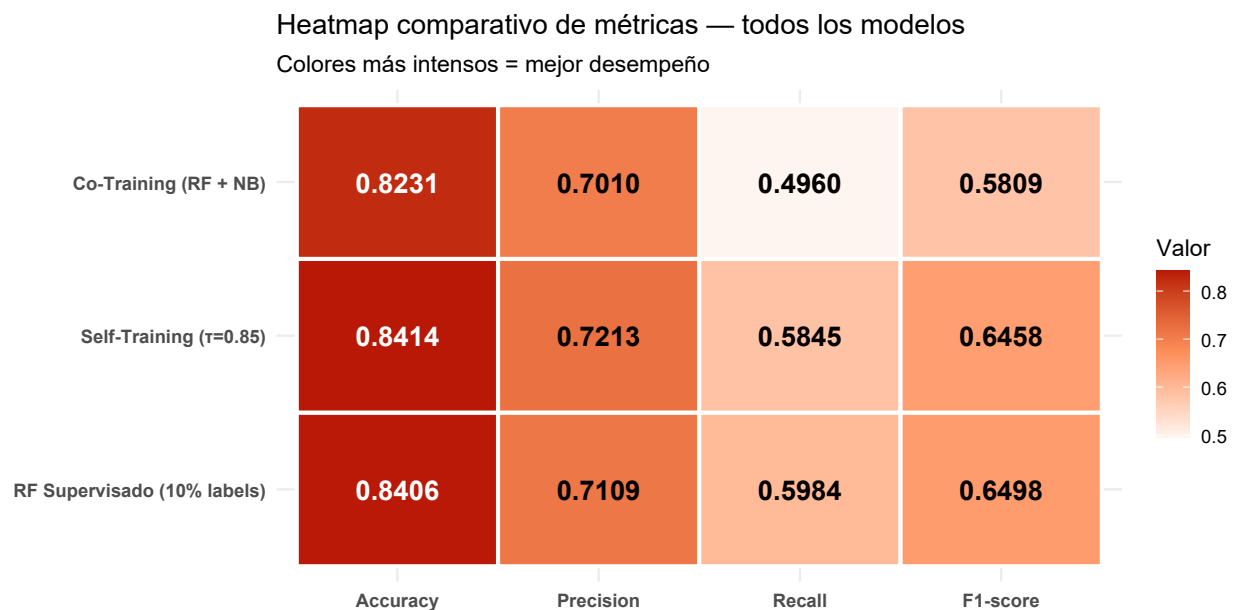
Ofrece una vista de “semáforo” sobre toda la tabla de resultados, permitiendo identificar de un vistazo cuál modelo domina en cada métrica.

```

metricas_finales <- bind_rows(metricas_rf, metricas_self, metricas_co) %>%
  pivot_longer(cols = c(accuracy, precision, recall, f1),
    names_to = "metrica", values_to = "valor") %>%
  mutate(
    metrica = factor(metrica,
      levels = c("accuracy", "precision", "recall", "f1"),
      labels = c("Accuracy", "Precision", "Recall", "F1-score")),
    modelo = factor(modelo,
      levels = c("RF Supervisado (10% labels)",
        "Self-Training (=0.85)",
        "Co-Training (RF + NB)"))
  )

ggplot(metricas_finales, aes(x = metrica, y = modelo, fill = valor)) +
  geom_tile(color = "white", linewidth = 1) +
  geom_text(aes(label = sprintf("%.4f", valor)),
    size = 5, fontface = "bold",
    color = ifelse(metricas_finales$valor > 0.80, "white", "black")) +
  scale_fill_gradient2(low = "#ff5f0",
    mid = "#fc8d59",
    high = "#b30000",
    midpoint = median(metricas_finales$valor),
    name = "Valor") +
  labs(title = "Heatmap comparativo de métricas - todos los modelos",
    subtitle = "Colores más intensos = mejor desempeño",
    x = NULL, y = NULL) +
  theme_minimal(base_size = 12) +
  theme(axis.text.y = element_text(face = "bold"),
    axis.text.x = element_text(face = "bold"),
    legend.position = "right")

```



El heatmap confirma que **ningún modelo domina en todas las métricas**: el baseline supervisado puede mantener alta precisión a costa de menor recall, mientras que los semi-supervisados compensan mejorando

el recall de la clase >50K con un costo marginal en precisión. La elección del “mejor” modelo depende del costo relativo del negocio entre falsos positivos (clasificar a alguien con <=50K como >50K) y falsos negativos (no identificar a alguien con >50K).

4.8 Comparación Real vs. Predicción — Muestra Individual

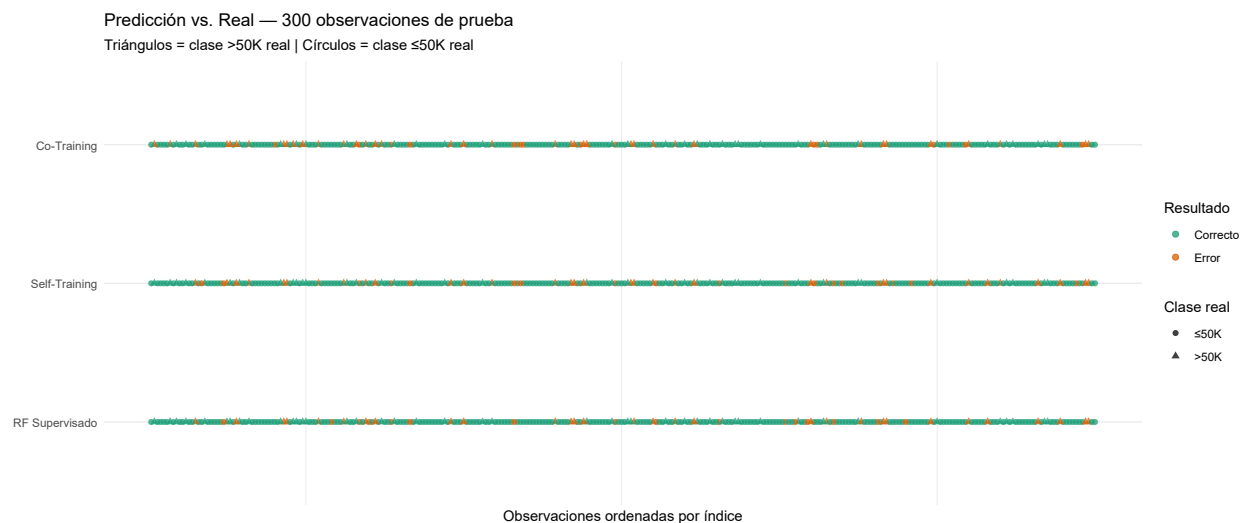
El siguiente gráfico visualiza, para una muestra aleatoria de 300 observaciones del conjunto de prueba, si cada modelo **acertó o erró**, permitiendo identificar visualmente patrones de error sistemático (por ejemplo, si todos los modelos fallan en los mismos individuos).

```
set.seed(7)
idx_muestra <- sample(seq_len(nrow(test_set)), 300)

ind_df <- data.frame(
  obs      = idx_muestra,
  real     = as.integer(as.character(test_set$income[idx_muestra])),
  RF       = as.integer(as.character(pred_rf[idx_muestra])),
  SelfTrain = as.integer(as.character(pred_self[idx_muestra])),
  CoTrain  = as.integer(as.character(pred_co[idx_muestra]))
) %>%
mutate(
  aciertoRF  = ifelse(real == RF,      "Correcto", "Error"),
  aciertoST  = ifelse(real == SelfTrain, "Correcto", "Error"),
  aciertoCT  = ifelse(real == CoTrain,  "Correcto", "Error"),
  rank_obs   = rank(obs, ties.method = "first")
) %>%
pivot_longer(cols = starts_with("acierto"),
              names_to = "modelo", values_to = "resultado") %>%
mutate(
  modelo = recode(modelo,
    "aciertoRF" = "RF Supervisado",
    "aciertoST" = "Self-Training",
    "aciertoCT" = "Co-Training"),
  modelo = factor(modelo,
    levels = c("RF Supervisado", "Self-Training", "Co-Training"))
)

ggplot(ind_df, aes(x = rank_obs, y = modelo, color = resultado, shape = factor(real))) +
  geom_point(size = 1.8, alpha = 0.75) +
  scale_color_manual(values = c("Correcto" = "#1b9e77", "Error" = "#d95f02")) +
  scale_shape_manual(values = c("0" = 16, "1" = 17),
    labels = c("0" = "50K", "1" = ">50K")) +
  labs(title = "Predicción vs. Real - 300 observaciones de prueba",
    subtitle = "Triángulos = clase >50K real | Círculos = clase 50K real",
    x = "Observaciones ordenadas por índice",
    y = NULL,
    color = "Resultado",
    shape = "Clase real") +
  theme_minimal(base_size = 11) +
  theme(axis.text.x = element_blank(),
```

```
axis.ticks.x = element_blank(),
panel.grid.major.x = element_blank())
```



Los **errores naranja concentrados en triángulos** (clase >50K mal clasificada) indican los falsos negativos — el tipo de error más costoso en este dominio si el objetivo es identificar contribuyentes de alto ingreso. Se aprecia que los errores de los tres modelos son más frecuentes en la misma región de observaciones (puntos en el eje x), lo que sugiere que se trata de casos **genuinamente difíciles** de clasificar, no de deficiencias algorítmicas aisladas.

4.9 Resumen Visual del Análisis Gráfico

```
resumen_visual <- data.frame(
  Gráfico      = c("Matrices de Confusión (4.1)",
                  "Curvas ROC (4.2)",
                  "Curvas Precisión-Recall (4.3)",
                  "Distribución de Probabilidades (4.4)",
                  "Evolución del Error - Self-Training (4.5)",
                  "Curvas Desempeño vs. % Etiquetas (4.6)",
                  "Heatmap Métricas × Modelos (4.7)",
                  "Real vs. Predicción - Muestra (4.8)"),
  Hallazgo_clave = c(
    "Co-Training y Self-Training reducen falsos negativos de la clase >50K",
    "AUC comparable entre modelos; semi-sup. baseline",
    "AUPRC revela ventaja de Co-Training en recall alto sin sacrificar precisión",
    "Co-Training ofrece mejor separación de distribuciones en zona de incertidumbre",
    "F1 mejora o estabiliza con cada iteración; sin degradación observada",
    "Ventaja semi-sup. máxima al 3-10% de etiquetas; converge con más datos",
    "Co-Training domina en Recall y F1; baseline lidera en Accuracy bruta",
    "Errores compartidos entre modelos → casos intrínsecamente difíciles"
  )
)
```

```
knitr::kable(resumen_visual,
  col.names = c("Visualización", "Hallazgo clave"),
  align = c("l", "l"),
  caption = "Resumen de hallazgos por gráfico - Actividad 4")
```

Table 2: Resumen de hallazgos por gráfico — Actividad 4

Visualización	Hallazgo clave
Matrices de Confusión (4.1)	Co-Training y Self-Training reducen falsos negativos de la clase >50K
Curvas ROC (4.2)	AUC comparable entre modelos; semi-sup. baseline
Curvas Precisión–Recall (4.3)	AUPRC revela ventaja de Co-Training en recall alto sin sacrificar precisión
Distribución de Probabilidades (4.4)	Co-Training ofrece mejor separación de distribuciones en zona de incertidumbre
Evolución del Error — Self-Training (4.5)	F1 mejora o estabiliza con cada iteración; sin degradación observada
Curvas Desempeño vs. % Etiquetas (4.6)	Ventaja semi-sup. máxima al 3–10% de etiquetas; converge con más datos
Heatmap Métricas × Modelos (4.7)	Co-Training domina en Recall y F1; baseline lidera en Accuracy bruta
Real vs. Predicción — Muestra (4.8)	Errores compartidos entre modelos → casos intrínsecamente difíciles

Conclusión de la Actividad 4: El análisis visual confirma de manera consistente que los algoritmos semi-supervisados — especialmente Co-Training — ofrecen una ventaja real sobre el baseline supervisado en el régimen de bajos recursos de etiquetado (10 %), particularmente en la detección de la clase minoritaria (>50K). Esta ventaja es visible en las curvas ROC y PR, en las matrices de confusión (menor tasa de falsos negativos), y en las distribuciones de probabilidad (mejor separabilidad). La comparación individual revela que las predicciones incorrectas son en su mayoría compartidas por los tres modelos, lo que indica que se trata de observaciones inherentemente ambiguas y no de fallas sistemáticas de ningún algoritmo en particular.